

The Ultimate R Basics Cheat Sheet

Parsa

2026-09-21

Contents

1	How to Use This Cheat Sheet	4
2	Part 0 — The 30-Second Survival Guide	5
3	Part 1 — Installing and Loading Packages	5
3.1	1.1 Install once	5
3.2	1.2 Load every session	6
3.3	1.3 See what's installed and loaded	6
3.4	1.4 Update	6
4	Part 2 — Getting Help	6
5	Part 3 — Variables and Assignment	7
5.1	3.1 Assignment operators	7
5.2	3.2 Naming rules	7
5.3	3.3 List and remove objects	7
5.4	3.4 Data types overview	7
6	Part 4 — Data Types in Detail	8
6.1	4.1 Numeric	8
6.2	4.2 Character	8
6.3	4.3 Logical	9
6.4	4.4 Coercion	9
6.5	4.5 Type check and convert	10
7	Part 5 — Data Structures	10
7.1	5.1 Vector (1D, same type)	10
7.2	5.2 Matrix (2D, same type)	10
7.3	5.3 Array (nD, same type)	11
7.4	5.4 List (1D, mixed types)	11
7.5	5.5 Data frame (2D, mixed types, columns same length)	12
7.6	5.6 Tibble (modern data frame)	13
7.7	5.7 Summary: which structure to use	13
8	Part 6 — Indexing and Subsetting	14
8.1	6.1 Vectors	14
8.2	6.2 Matrices	14
8.3	6.3 Data frames / tibbles	15
8.4	6.4 Lists	16
8.5	6.5 \$ vs [[vs [.	16
8.6	6.6 Named vectors	16

9	Part 7 — Operators	17
9.1	7.1 Arithmetic	17
9.2	7.2 Comparison	18
9.3	7.3 Logical	18
9.4	7.4 Membership and matching	19
9.5	7.5 Pipe operators	19
9.6	7.6 Special values	19
10	Part 8 — The Pipe in Depth	20
10.1	8.1 Why pipe	20
10.2	8.2 Pipe with data frames	21
10.3	8.3 Common pipe mistake	21
11	Part 9 — Functions	21
11.1	9.1 Calling functions	21
11.2	9.2 Common math/stat functions	22
11.3	9.3 Writing your own function	22
11.4	9.4 Anonymous functions	23
11.5	9.5 ... (dots)	23
11.6	9.6 Scope	24
12	Part 10 — Control Flow	24
12.1	10.1 if / else	24
12.2	10.2 for loop	25
12.3	10.3 while loop	25
12.4	10.4 break and next	25
12.5	10.5 Vectorization (preferred over loops)	25
12.6	10.6 The apply family	26
13	Part 11 — Strings	27
13.1	11.1 Basics	27
13.2	11.2 Pattern matching	27
13.3	11.3 stringr (tidyverse)	28
13.4	11.4 Printing strings	28
13.5	11.5 Glue	29
14	Part 12 — Factors	29
14.1	12.1 Creating factors	29
14.2	12.2 Setting level order	29
14.3	12.3 Changing the reference level	30
14.4	12.4 Table and counts	30
14.5	12.5 Convert to/from factor	30
14.6	12.6 forcats (tidyverse)	30
15	Part 13 — Dates	31
15.1	13.1 Creating dates	31
15.2	13.2 Components	31
15.3	13.3 Arithmetic	32
15.4	13.4 Sequences	32
15.5	13.5 lubridate	33
16	Part 14 — Missing Values	33
16.1	14.1 Detecting	33
16.2	14.2 Removing	34
16.3	14.3 Functions with <code>na.rm</code>	34

16.4	14.4	Filling / replacing	34
16.5	14.5	NA in data frames	35
16.6	14.6	NA vs NULL vs NaN	35
17		Part 15 — Reading and Writing Files	36
17.1	15.1	CSV	36
17.2	15.2	Excel	36
17.3	15.3	RDS (R’s native format — preserves types)	36
17.4	15.4	RData (multiple objects)	36
17.5	15.5	Text files	36
17.6	15.6	From clipboard	36
17.7	15.7	Inspect files	36
18		Part 16 — Data Manipulation with dplyr	38
18.1	16.1	The five verbs	38
18.2	16.2	filter	39
18.3	16.3	select	40
18.4	16.4	mutate	41
18.5	16.5	arrange	42
18.6	16.6	summarise + group_by	42
18.7	16.7	count and tally	43
18.8	16.8	slice and distinct	43
18.9	16.9	pull and rename	45
18.10	16.10	across (apply to many columns)	45
18.11	16.11	Summary of dplyr verbs	45
19		Part 17 — Reshaping with tidyr	46
19.1	17.1	pivot_longer (wide → long)	46
19.2	17.2	pivot_wider (long → wide)	47
19.3	17.3	separate and unite	47
19.4	17.4	Complete and fill	48
20		Part 18 — Joins	48
20.1	18.1	The four main joins	48
20.2	18.2	Anti and semi joins	49
20.3	18.3	Joining by different names	50
20.4	18.4	Join summary	50
21		Part 19 — purrr (Functional Programming)	50
21.1	19.1	map	50
21.2	19.2	map over data frame columns	51
21.3	19.3	map2 and pmap	51
21.4	19.4	walk (for side effects)	51
22		Part 20 — Base R Plots	51
22.1	20.1	Scatter plot	51
22.2	20.2	Line plot	52
22.3	20.3	Histogram	53
22.4	20.4	Boxplot	54
22.5	20.5	Bar plot	55
22.6	20.6	Layout: multiple plots	56
22.7	20.7	Save a plot	57
23		Part 21 — ggplot2	57
23.1	21.1	Grammar of graphics	57

23.2	21.2	Scatter plot	58
23.3	21.3	Line plot	58
23.4	21.4	Histogram	59
23.5	21.5	Boxplot	60
23.6	21.6	Bar plot	61
23.7	21.7	Facets	62
23.8	21.8	Saving	63
23.9	21.9	Common geoms	63
23.10	21.10	Common theme tweaks	63
24	Part 22	— Statistical Functions	64
24.1	22.1	Descriptive	64
24.2	22.2	Distributions	64
24.3	22.3	Hypothesis tests	65
24.4	22.4	Regression	66
24.5	22.5	Correlation	67
24.6	22.6	Hypothesis test decision table	68
25	Part 23	— Random Numbers and Sampling	68
25.1	23.1	Setting the seed	68
25.2	23.2	Common generators	68
25.3	23.3	Sampling	69
25.4	23.4	Split into train/test	69
26	Part 24	— Vectorization vs Loops	70
26.1	24.1	Why vectorize	70
26.2	24.2	Vectorized operations	70
26.3	24.3	Recycling	71
26.4	24.4	When to use loops	71
27	Part 25	— Debugging	71
27.1	25.1	Errors vs warnings vs messages	71
27.2	25.2	Reading error messages	71
27.3	25.3	Debugging tools	71
27.4	25.4	Common checks	72
27.5	25.5	stopifnot	73
28	Part 26	— Common Errors and Fixes	73
28.1	26.1	Quick fixes	73
29	Part 27	— RStudio Shortcuts	74
30	Part 28	— Quick Reference Table	74
31	Part 29	— Theory Q&A (What to Say)	75
32	Part 30	— The Single Most Important Idea	76
32.1		What’s inside and why	76

1 How to Use This Cheat Sheet

This is a complete R reference. Every section is independent. Find what you need, copy the pattern, change the names.

Topic	Section
Assign variables	3
Know data types	4
Build vectors, lists, data frames	5
Subset data with <code>[], [[, \$</code>	6
Arithmetic, comparison, logic	7
Pipe <code> ></code> and <code>%>%</code>	8
Write functions	9
If/else, loops, apply family	10
Strings	11
Factors	12
Dates	13
Missing values	14
Read/write files	15
dplyr: filter, select, mutate, arrange	16
Grouped summaries	17
tidyr: pivot, separate, unite	18
Joins	19
purrr: map family	20
Base R plots	21
ggplot2 plots	22
Statistical functions	23
Random numbers	24
Vectorization vs loops	25
Debugging	26
Common errors	27
Packages	28
RStudio shortcuts	29

2 Part 0 — The 30-Second Survival Guide

If you only remember five things:

1. **Assign with `<-`:** `x <- 5`.
2. **Call functions with parentheses:** `mean(x)`.
3. **Index with square brackets:** `x[1]`, `df[1, 2]`, `df$col`.
4. **Pipe chains operations:** `x |> mean()` `|> round(2)`.
5. **Get help with `?:`** `?mean`.

Everything else is syntax you can look up.

3 Part 1 — Installing and Loading Packages

3.1 1.1 Install once

```
install.packages("tidyverse")
install.packages(c("forecast", "car", "lmtest"))
```

3.2 1.2 Load every session

```
library(tidyverse)
library(forecast)
```

Rules: - `install.packages()` needs quotes around the name. - `library()` does **not** need quotes. - You only install once; you load every session.

3.3 1.3 See what's installed and loaded

```
# All installed packages
length(rownames(installed.packages()))
```

```
## [1] 209
```

```
# Currently loaded
search()
```

```
## [1] ".GlobalEnv"      "package:knitr"      "package:lubridate"
## [4] "package:forcats"  "package:stringr"    "package:dplyr"
## [7] "package:purrr"    "package:readr"      "package:tidyr"
## [10] "package:tibble"   "package:ggplot2"    "package:tidyverse"
## [13] "package:stats"    "package:graphics"   "package:grDevices"
## [16] "package:utils"    "package:datasets"   "package:methods"
## [19] "Autoloads"        "package:base"
```

3.4 1.4 Update

```
update.packages(ask = FALSE)
```

4 Part 2 — Getting Help

What you want	How
Function help	?mean
Function help (alt)	help("mean")
Search help	??regression
See arguments	args(lm)
See examples	example(lm)
See source code	lm (just type the name, no parens)
Vignettes	vignette("dplyr")
Package help	help(package = "dplyr")

```
# --- Self-contained: help examples ---
?mean
args(lm)
```

```
## function (formula, data, subset, weights, na.action, method = "qr",
##      model = TRUE, x = FALSE, y = FALSE, qr = TRUE, singular.ok = TRUE,
##      contrasts = NULL, offset, ...)
## NULL
```

Tip: In RStudio, highlight a function name and press **F1** to open help.

5 Part 3 — Variables and Assignment

5.1 3.1 Assignment operators

```
# --- Self-contained: assignment ---  
p3_x <- 5          # standard  
p3_y = 10          # works, but not idiomatic  
15 -> p3_z         # right-to-left (rare)
```

```
p3_x
```

```
## [1] 5
```

```
p3_y
```

```
## [1] 10
```

```
p3_z
```

```
## [1] 15
```

Use `<-`. Reserve `=` for function arguments.

5.2 3.2 Naming rules

- Start with a letter (or `.` not followed by a digit).
- Can contain letters, digits, `.`, `_`.
- **Case-sensitive:** `x` and `X` are different.
- Avoid `c`, `T`, `F`, `mean` — they're built-in.

Valid: `my_var`, `x1`, `myVar`, `data.2024` Invalid: `1x`, `my-var`, `my var`

5.3 3.3 List and remove objects

```
# --- Self-contained: list and remove objects ---  
p3_a <- 1  
p3_b <- 2  
p3_c <- 3
```

```
ls()          # list objects (in this session, many exist)
```

```
## [1] "p3_a" "p3_b" "p3_c" "p3_x" "p3_y" "p3_z"
```

```
ls(pattern = "^p3_")  # list by pattern
```

```
## [1] "p3_a" "p3_b" "p3_c" "p3_x" "p3_y" "p3_z"
```

```
rm(p3_a)        # remove one
```

```
rm(list = ls(pattern = "^p3_"))  # remove all p3_ objects
```

5.4 3.4 Data types overview

Type	Example	Check with
numeric (double)	3.14, -1.5e3	is.numeric(x)
integer	5L, -2L	is.integer(x)
character	"hello", 'hi'	is.character(x)
logical	TRUE, FALSE, T, F	is.logical(x)
complex	3 + 2i	is.complex(x)
factor	factor(c("a", "b"))	is.factor(x)
Date	as.Date("2024-01-01")	is.Date(x)

6 Part 4 — Data Types in Detail

6.1 4.1 Numeric

```
# --- Self-contained: numeric ---
p4_a <- 3.14
p4_b <- 1e3      # 1000
p4_c <- -2.5
p4_d <- 5L      # integer (add L)

class(p4_a)      # "numeric"
```

```
## [1] "numeric"
```

```
class(p4_d)      # "integer"
```

```
## [1] "integer"
```

```
is.numeric(p4_a) # TRUE
```

```
## [1] TRUE
```

```
is.integer(p4_a) # FALSE
```

```
## [1] FALSE
```

Note: By default, numbers are `numeric` (double). Use `L` suffix for integers.

6.2 4.2 Character

```
# --- Self-contained: character ---
p4_name <- "Parsa"
p4_greeting <- 'Hello'
p4_sentence <- "She said \"hi\""

nchar(p4_name)      # number of characters
```

```
## [1] 5
```

```
toupper(p4_name)
```

```
## [1] "PARSA"
```

```
tolower(p4_name)
```

```
## [1] "parsa"
```

```
paste(p4_greeting, p4_name)
```

```
## [1] "Hello Parsa"
```

```
paste0(p4_greeting, " ", p4_name, "!")
```

```
## [1] "Hello, Parsa!"
```

Rule: Single and double quotes are equivalent. Use `\` to escape quotes inside a string.

6.3 4.3 Logical

```
# --- Self-contained: logical ---
```

```
p4_t <- TRUE
```

```
p4_f <- FALSE
```

```
p4_cmp <- 3 > 2      # TRUE
```

```
p4_neg <- !p4_t      # FALSE
```

```
p4_t & p4_f          # AND -> FALSE
```

```
## [1] FALSE
```

```
p4_t | p4_f          # OR -> TRUE
```

```
## [1] TRUE
```

```
p4_t && p4_f          # AND (scalar) -> FALSE
```

```
## [1] FALSE
```

```
p4_t || p4_f          # OR (scalar) -> TRUE
```

```
## [1] TRUE
```

`&` and `|` are vectorized; `&&` and `||` short-circuit and return one value.

6.4 4.4 Coercion

R converts between types when forced. Order of precedence: **logical** < **integer** < **numeric** < **character**.

```
# --- Self-contained: coercion ---
```

```
p4_mixed <- c(1, 2, "three") # all become character
```

```
p4_mixed
```

```
## [1] "1"      "2"      "three"
```

```
class(p4_mixed)
```

```
## [1] "character"
```

```
as.numeric("3.14")      # 3.14
```

```
## [1] 3.14
```

```
as.numeric("abc")        # NA with warning
```

```
## [1] NA
```

```
as.character(42)         # "42"
```

```
## [1] "42"
```

```
as.logical(0)                # FALSE
```

```
## [1] FALSE
```

```
as.logical(1)                # TRUE
```

```
## [1] TRUE
```

Force conversion explicitly with `as.*()` to avoid surprises.

6.5 4.5 Type check and convert

```
# --- Self-contained: type checks ---
```

```
p4_v <- c(1, 2, 3)
```

```
is.numeric(p4_v)
```

```
## [1] TRUE
```

```
is.character(p4_v)
```

```
## [1] FALSE
```

```
is.vector(p4_v)
```

```
## [1] TRUE
```

```
as.character(p4_v)
```

```
## [1] "1" "2" "3"
```

7 Part 5 — Data Structures

7.1 5.1 Vector (1D, same type)

```
# --- Self-contained: vectors ---
```

```
p5_num <- c(1, 2, 3, 4, 5)
```

```
p5_chr <- c("a", "b", "c")
```

```
p5_lgl <- c(TRUE, FALSE, TRUE)
```

```
p5_seq <- 1:10
```

```
p5_rep <- rep(1:3, times = 2)    # 1 2 3 1 2 3
```

```
p5_rep2 <- rep(1:3, each = 2)   # 1 1 2 2 3 3
```

```
p5_seq2 <- seq(0, 1, by = 0.25) # 0.00 0.25 0.50 0.75 1.00
```

```
length(p5_num)
```

```
## [1] 5
```

```
length(p5_chr)
```

```
## [1] 3
```

7.2 5.2 Matrix (2D, same type)

```

# --- Self-contained: matrices ---
p5_mat <- matrix(1:12, nrow = 3, ncol = 4)
p5_mat

##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12

p5_mat2 <- matrix(1:12, nrow = 3, byrow = TRUE)
p5_mat2

##      [,1] [,2] [,3] [,4]
## [1,]    1    2    3    4
## [2,]    5    6    7    8
## [3,]    9   10   11   12

dim(p5_mat)      # 3 4

## [1] 3 4
nrow(p5_mat); ncol(p5_mat)

## [1] 3
## [1] 4

t(p5_mat)        # transpose

##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
## [4,]   10   11   12

p5_mat %*% t(p5_mat) # matrix multiplication

##      [,1] [,2] [,3]
## [1,]  166  188  210
## [2,]  188  214  240
## [3,]  210  240  270

```

7.3 5.3 Array (nD, same type)

```

# --- Self-contained: arrays ---
p5_arr <- array(1:24, dim = c(2, 3, 4))
dim(p5_arr)

## [1] 2 3 4

p5_arr[1, 2, 3]      # one element

## [1] 15

```

7.4 5.4 List (1D, mixed types)

```

# --- Self-contained: lists ---
p5_list <- list(

```

```

  name = "Parsa",
  age   = 25,
  scores = c(90, 85, 88),
  passed = TRUE
)
p5_list

```

```

## $name
## [1] "Parsa"
##
## $age
## [1] 25
##
## $scores
## [1] 90 85 88
##
## $passed
## [1] TRUE

```

```
p5_list$name
```

```
## [1] "Parsa"
```

```
p5_list[["age"]]
```

```
## [1] 25
```

```
p5_list[[3]]           # the scores vector
```

```
## [1] 90 85 88
```

```
length(p5_list)
```

```
## [1] 4
```

```
names(p5_list)
```

```
## [1] "name"  "age"   "scores" "passed"
```

7.5 5.5 Data frame (2D, mixed types, columns same length)

```

# --- Self-contained: data frame ---
p5_df <- data.frame(
  id   = 1:4,
  name = c("A", "B", "C", "D"),
  score = c(90, 85, 88, 92)
)
p5_df

```

```

##   id name score
## 1  1   A    90
## 2  2   B    85
## 3  3   C    88
## 4  4   D    92

```

```
str(p5_df)
```

```
## 'data.frame':   4 obs. of  3 variables:
```

```
## $ id : int 1 2 3 4
## $ name : chr "A" "B" "C" "D"
## $ score: num 90 85 88 92
```

```
summary(p5_df)
```

```
##           id           name           score
##  Min.      :1.00   Length   :4   Min.      :85.00
##  1st Qu.:1.75   N.unique  :4   1st Qu.:87.25
##  Median :2.50   N.blank   :0   Median :89.00
##  Mean    :2.50   Min.nchar:1   Mean    :88.75
##  3rd Qu.:3.25   Max.nchar:1   3rd Qu.:90.50
##  Max.    :4.00                      Max.    :92.00
```

```
nrow(p5_df); ncol(p5_df)
```

```
## [1] 4
```

```
## [1] 3
```

```
names(p5_df)
```

```
## [1] "id"      "name"    "score"
```

7.6 5.6 Tibble (modern data frame)

```
# --- Self-contained: tibble ---
p5_tib <- tibble(
  id    = 1:4,
  name  = c("A", "B", "C", "D"),
  score = c(90, 85, 88, 92)
)
p5_tib
```

```
## # A tibble: 4 x 3
##       id name  score
##   <int> <chr> <dbl>
## 1     1  A      90
## 2     2  B      85
## 3     3  C      88
## 4     4  D      92
```

```
class(p5_tib)
```

```
## [1] "tbl_df"      "tbl"        "data.frame"
```

Tibble vs data.frame: - Tibbles print nicely (no row-name clutter, only first 10 rows shown). - Tibbles don't auto-convert strings to factors (R 4.0). - Tibbles allow list-columns.

7.7 5.7 Summary: which structure to use

Need	Use
One column of data	vector
Table of same-type numbers (math)	matrix
Multi-dimensional numeric array	array
Mixed-type container	list
Table with named columns	tibble (or data.frame)

8 Part 6 — Indexing and Subsetting

8.1 6.1 Vectors

```
# --- Self-contained: vector indexing ---
p6_v <- c(10, 20, 30, 40, 50)

p6_v[1]          # 10 (first)

## [1] 10
p6_v[c(1, 3)]    # 10 30

## [1] 10 30
p6_v[-2]         # remove 2nd -> 10 30 40 50

## [1] 10 30 40 50
p6_v[2:4]        # 20 30 40

## [1] 20 30 40
p6_v[p6_v > 25]  # 30 40 50

## [1] 30 40 50
p6_v[c(TRUE, FALSE, TRUE, FALSE, TRUE)] # 10 30 50

## [1] 10 30 50
```

Warning: R indexing starts at 1, not 0.

8.2 6.2 Matrices

```
# --- Self-contained: matrix indexing ---
p6_m <- matrix(1:12, nrow = 3)
p6_m

##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
p6_m[1, 1]       # row 1, column 1

## [1] 1
p6_m[1, ]        # row 1

## [1] 1  4  7 10
p6_m[, 2]        # column 2

## [1] 4 5 6
p6_m[1:2, 2:3]   # submatrix
```

```
##      [,1] [,2]
## [1,]    4    7
## [2,]    5    8
```

```
p6_m[p6_m > 5]      # vector of elements > 5
```

```
## [1]  6  7  8  9 10 11 12
```

8.3 6.3 Data frames / tibbles

```
# --- Self-contained: data frame indexing ---
```

```
p6_df <- tibble(
  id   = 1:4,
  name = c("A", "B", "C", "D"),
  score = c(90, 85, 88, 92)
)
p6_df
```

```
## # A tibble: 4 x 3
##       id name  score
##   <int> <chr> <dbl>
## 1     1  A     90
## 2     2  B     85
## 3     3  C     88
## 4     4  D     92
```

```
p6_df[1, ]      # row 1
```

```
## # A tibble: 1 x 3
##       id name  score
##   <int> <chr> <dbl>
## 1     1  A     90
```

```
p6_df[, "score"]      # column by name (returns tibble)
```

```
## # A tibble: 4 x 1
##       score
##   <dbl>
## 1     90
## 2     85
## 3     88
## 4     92
```

```
p6_df$score      # column by $ (returns vector)
```

```
## [1] 90 85 88 92
```

```
p6_df[["score"]]      # column by [[ (returns vector)
```

```
## [1] 90 85 88 92
```

```
p6_df$score[2]      # 2nd element of score
```

```
## [1] 85
```

```
p6_df[p6_df$score > 87, ]      # rows where score > 87
```

```
## # A tibble: 3 x 3
##       id name  score
```

```
##    <int> <chr> <dbl>
## 1      1 A      90
## 2      3 C      88
## 3      4 D      92
```

8.4 6.4 Lists

```
# --- Self-contained: list indexing ---
p6_l <- list(a = 1, b = 2:4, c = "hi")

p6_l[1]          # sub-list with one element

## $a
## [1] 1
p6_l[[1]]        # the element itself (numeric 1)

## [1] 1
p6_l$b           # the vector 2:4

## [1] 2 3 4
p6_l[["b"]]      # same

## [1] 2 3 4
p6_l$b[2]        # 3

## [1] 3
```

Rule: `[` returns a container, `[[` returns the contents.

8.5 6.5 \$ vs [[vs [

Access	Returns	Use for
<code>df\$col</code>	vector	one column by name
<code>df[["col"]]</code>	vector	one column, programmatic
<code>df["col"]</code>	tibble	one column, keep as tibble
<code>df[1]</code>	tibble	first column
<code>df[1,]</code>	tibble	first row
<code>df[[1]]</code>	vector	first column, contents

8.6 6.6 Named vectors

```
# --- Self-contained: named vectors ---
p6_named <- c(a = 1, b = 2, c = 3)
p6_named["b"]      # 2

## b
## 2
p6_named[["b"]]    # 2

## [1] 2
```

```
names(p6_named)  # "a" "b" "c"
```

```
## [1] "a" "b" "c"
```

9 Part 7 — Operators

9.1 7.1 Arithmetic

```
# --- Self-contained: arithmetic ---  
7 + 3      # 10
```

```
## [1] 10
```

```
7 - 3      # 4
```

```
## [1] 4
```

```
7 * 3      # 21
```

```
## [1] 21
```

```
7 / 3      # 2.333...
```

```
## [1] 2.333333
```

```
7 %/% 3    # 2 (integer division)
```

```
## [1] 2
```

```
7 %% 3     # 1 (modulo)
```

```
## [1] 1
```

```
7 ^ 3      # 343
```

```
## [1] 343
```

```
sqrt(16)   # 4
```

```
## [1] 4
```

```
abs(-5)    # 5
```

```
## [1] 5
```

```
exp(1)     # 2.718...
```

```
## [1] 2.718282
```

```
log(100)   # natural log
```

```
## [1] 4.60517
```

```
log10(100) # 2
```

```
## [1] 2
```

```
log2(8)    # 3
```

```
## [1] 3
```

9.2 7.2 Comparison

```
# --- Self-contained: comparison ---  
3 == 3      # TRUE
```

```
## [1] TRUE
```

```
3 != 4      # TRUE
```

```
## [1] TRUE
```

```
3 < 4       # TRUE
```

```
## [1] TRUE
```

```
3 <= 4      # TRUE
```

```
## [1] TRUE
```

```
3 > 4       # FALSE
```

```
## [1] FALSE
```

```
3 >= 4      # FALSE
```

```
## [1] FALSE
```

```
"a" < "b"   # TRUE (alphabetical)
```

```
## [1] TRUE
```

Warning: == is comparison, = is assignment.

9.3 7.3 Logical

```
# --- Self-contained: logical ---  
TRUE & TRUE  # TRUE
```

```
## [1] TRUE
```

```
TRUE & FALSE # FALSE
```

```
## [1] FALSE
```

```
TRUE | FALSE # TRUE
```

```
## [1] TRUE
```

```
!TRUE       # FALSE
```

```
## [1] FALSE
```

```
# Short-circuit (scalar, stops early)
```

```
TRUE && FALSE # FALSE
```

```
## [1] FALSE
```

```
TRUE || FALSE # TRUE
```

```
## [1] TRUE
```

```
# Vectorized
```

```
c(TRUE, FALSE) & c(TRUE, TRUE) # TRUE FALSE
```

```
## [1] TRUE FALSE
```

9.4 7.4 Membership and matching

```
# --- Self-contained: membership ---
3 %in% c(1, 2, 3)      # TRUE

## [1] TRUE

"a" %in% c("a", "b")   # TRUE

## [1] TRUE

c(1, 2, 3) %in% c(2, 3) # FALSE TRUE TRUE

## [1] FALSE TRUE TRUE

match(c("b", "a"), c("a", "b", "c")) # 2 1

## [1] 2 1
```

9.5 7.5 Pipe operators

```
# --- Self-contained: pipes ---
set.seed(701)
p7_v <- rnorm(10)

# Without pipe
round(mean(p7_v), 2)
```

```
## [1] -0.18
```

```
# Base pipe (R >= 4.1)
p7_v |> mean() |> round(2)
```

```
## [1] -0.18
```

```
# magrittr pipe (tidyverse)
p7_v %>% mean() %>% round(2)
```

```
## [1] -0.18
```

Rule: `x |> f()` is the same as `f(x)`. Pipes pass the left side as the **first** argument of the right side.

Placeholder for non-first argument:

```
# --- Self-contained: pipe with placeholder ---
p7_x <- c(1, NA, 3)
p7_x |> mean(na.rm = TRUE) # works
```

```
## [1] 2
```

```
# p7_x |> mean(x = _)      # _ only with base pipe, R >= 4.2
```

9.6 7.6 Special values

```
# --- Self-contained: special values ---
Inf      # positive infinity
```

```
## [1] Inf
```

```

-Inf
## [1] -Inf
NaN      # Not a Number (0/0)
## [1] NaN
NA        # missing
## [1] NA
NULL      # empty / no value
## NULL
1/0       # Inf
## [1] Inf
0/0       # NaN
## [1] NaN
NA + 1    # NA
## [1] NA
is.na(NA) # TRUE
## [1] TRUE
is.nan(NaN) # TRUE
## [1] TRUE
is.null(NULL) # TRUE
## [1] TRUE

```

10 Part 8 — The Pipe in Depth

10.1 8.1 Why pipe

Without pipe:

```

# --- Self-contained: without pipe ---
set.seed(801)
p8_v <- rnorm(20)
round(sqrt(mean(p8_v^2)), 3)

```

```
## [1] 1.102
```

With pipe:

```

# --- Self-contained: with pipe ---
set.seed(802)
p8_v <- rnorm(20)
p8_v |> (\(x) x^2)() |> mean() |> sqrt() |> round(3)

```

```
## [1] 0.741
```

The pipe helps readability when operations are nested.

10.2 8.2 Pipe with data frames

```
# --- Self-contained: pipe with dplyr ---
set.seed(803)
p8_df <- tibble(x = rnorm(20), y = rnorm(20))

p8_df |>
  filter(x > 0) |>
  mutate(z = x + y) |>
  summarise(mean_z = mean(z))
```

```
## # A tibble: 1 x 1
##   mean_z
##   <dbl>
## 1  0.590
```

10.3 8.3 Common pipe mistake

```
# --- WRONG ---
# p8_df |> filter(x > 0) |> lm(y ~ x) # lm expects data = first arg, but gets filtered df

# --- RIGHT ---
p8_df |> filter(x > 0) |> (\(d) lm(y ~ x, data = d))()
```

Rule: Pipe passes left side as **first argument**. If you need a different argument, wrap the function in an anonymous function `\(x) ...` (R 4.1) or `function(x) ....`

11 Part 9 — Functions

11.1 9.1 Calling functions

```
# --- Self-contained: calling functions ---
mean(c(1, 2, 3)) # 2

## [1] 2
mean(x = c(1, 2, 3)) # same, named arg

## [1] 2
round(3.14159, digits = 2) # 3.14

## [1] 3.14
sample(1:10, size = 3, replace = TRUE)

## [1] 5 10 1
```

Named arguments can be in any order. **Positional arguments** must be in order.

11.2 9.2 Common math/stat functions

```
# --- Self-contained: common functions ---
p9_x <- c(3, 1, 4, 1, 5, 9, 2, 6)

mean(p9_x)

## [1] 3.875
median(p9_x)

## [1] 3.5
sd(p9_x)          # standard deviation

## [1] 2.748376
var(p9_x)         # variance

## [1] 7.553571
min(p9_x)

## [1] 1
max(p9_x)

## [1] 9
range(p9_x)       # min and max

## [1] 1 9
sum(p9_x)

## [1] 31
prod(p9_x)

## [1] 6480
length(p9_x)

## [1] 8
quantile(p9_x, c(0.25, 0.5, 0.75))

## 25% 50% 75%
## 1.75 3.50 5.25
summary(p9_x)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.000   1.750   3.500   3.875   5.250   9.000
```

11.3 9.3 Writing your own function

```
# --- Self-contained: user function ---
p9_add <- function(a, b) {
  a + b
}
p9_add(2, 3)
```

```
## [1] 5
p9_square <- function(x) x^2
p9_square(4)

## [1] 16
# With default argument
p9_power <- function(x, p = 2) x^p
p9_power(3)      # 9

## [1] 9
p9_power(3, 3)   # 27

## [1] 27
# Returning multiple values (list)
p9_stats <- function(x) {
  list(mean = mean(x), sd = sd(x), n = length(x))
}
p9_stats(p9_x)

## $mean
## [1] 3.875
##
## $sd
## [1] 2.748376
##
## $n
## [1] 8
```

11.4 9.4 Anonymous functions

```
# --- Self-contained: anonymous functions ---
# R >= 4.1
p9_result1 <- (\(x) x^2)(5)
p9_result1
```

```
## [1] 25
# Classic
p9_result2 <- (function(x) x^2)(5)
p9_result2
```

```
## [1] 25
# With sapply
sapply(1:3, \(x) x^2)
```

```
## [1] 1 4 9
```

11.5 9.5 ... (dots)

Pass extra arguments through:

```
# --- Self-contained: dots ---
p9_mean_safe <- function(x, ...) {
  mean(x, ...)
```

```
}
p9_mean_safe(c(1, NA, 3), na.rm = TRUE)
```

```
## [1] 2
```

11.6 9.6 Scope

```
# --- Self-contained: scope ---
p9_global <- 10

p9_test <- function() {
  p9_local <- 5
  p9_global + p9_local  # can read global
}
p9_test()
```

```
## [1] 15
```

```
# p9_local  # ERROR: not found (it's local to p9_test)
```

Rule: Functions can read globals but cannot modify them without `<<-`.

12 Part 10 — Control Flow

12.1 10.1 if / else

```
# --- Self-contained: if/else ---
p10_x <- 7

if (p10_x > 5) {
  p10_result <- "big"
} else if (p10_x > 2) {
  p10_result <- "medium"
} else {
  p10_result <- "small"
}
p10_result
```

```
## [1] "big"
```

Vectorized version: `ifelse(test, yes, no)`.

```
# --- Self-contained: ifelse ---
p10_v <- c(-1, 0, 1, 2)
ifelse(p10_v > 0, "pos", "non-pos")
```

```
## [1] "non-pos" "non-pos" "pos"      "pos"
```

Note: `ifelse` evaluates both branches. For type-stable, use `dplyr::if_else()`.

```
# --- Self-contained: dplyr::if_else ---
p10_v2 <- c(-1, 0, 1, 2)
dplyr::if_else(p10_v2 > 0, "pos", "non-pos")
```

```
## [1] "non-pos" "non-pos" "pos"      "pos"
```

12.2 10.2 for loop

```
# --- Self-contained: for loop ---
p10_sum <- 0
for (i in 1:5) {
  p10_sum <- p10_sum + i
}
p10_sum
```

```
## [1] 15
```

```
# Loop over a vector
p10_fruits <- c("apple", "banana", "cherry")
for (f in p10_fruits) {
  cat("Fruit:", f, "\n")
}
```

```
## Fruit: apple
## Fruit: banana
## Fruit: cherry
```

12.3 10.3 while loop

```
# --- Self-contained: while loop ---
p10_i <- 1
while (p10_i <= 5) {
  cat("i =", p10_i, "\n")
  p10_i <- p10_i + 1
}
```

```
## i = 1
## i = 2
## i = 3
## i = 4
## i = 5
```

12.4 10.4 break and next

```
# --- Self-contained: break and next ---
for (i in 1:10) {
  if (i == 3) next # skip 3
  if (i == 7) break # stop at 7
  cat(i, " ")
}
```

```
## 1 2 4 5 6
```

12.5 10.5 Vectorization (preferred over loops)

```
# --- Self-contained: vectorization beats loops ---
set.seed(1001)
p10_v <- rnorm(1e6)

# Bad: loop
p10_slow <- numeric(length(p10_v))
```

```

for (i in seq_along(p10_v)) p10_slow[i] <- p10_v[i] * 2

# Good: vectorized
p10_fast <- p10_v * 2

identical(p10_slow, p10_fast)

## [1] TRUE

```

12.6 10.6 The apply family

```

# --- Self-contained: apply family ---
p10_mat <- matrix(1:12, nrow = 3)

# apply: matrix -> vector
apply(p10_mat, 1, sum) # row sums

## [1] 22 26 30

apply(p10_mat, 2, sum) # column sums

## [1] 6 15 24 33

# lapply: list/vector -> list
p10_list <- list(a = 1:3, b = 4:6, c = 7:9)
lapply(p10_list, sum)

## $a
## [1] 6
##
## $b
## [1] 15
##
## $c
## [1] 24

# sapply: list/vector -> vector (when possible)
sapply(p10_list, sum)

## a b c
## 6 15 24

# vapply: like sapply but type-safe
vapply(p10_list, sum, numeric(1))

## a b c
## 6 15 24

# tapply: split-apply-combine on groups
p10_groups <- c("A", "A", "B", "B", "A")
p10_values <- c(1, 2, 3, 4, 5)
tapply(p10_values, p10_groups, mean)

##      A      B
## 2.666667 3.500000

# mapply: multivariate apply
mapply(function(x, y) x + y, 1:3, 4:6)

```

```
## [1] 5 7 9
```

13 Part 11 — Strings

13.1 11.1 Basics

```
# --- Self-contained: strings ---
p11_s <- "Hello, World!"

nchar(p11_s)          # 13

## [1] 13
toupper(p11_s)

## [1] "HELLO, WORLD!"
tolower(p11_s)

## [1] "hello, world!"
substr(p11_s, 1, 5)    # "Hello"

## [1] "Hello"
substring(p11_s, 8)    # "World!"

## [1] "World!"
strsplit(p11_s, ", ")[[1]] # c("Hello", "World!")

## [1] "Hello" "World!"
paste("a", "b", "c")    # "a b c"

## [1] "a b c"
paste0("a", "b", "c")   # "abc"

## [1] "abc"
paste(c("a", "b"), 1:2, sep = "-") # "a-1" "b-2"

## [1] "a-1" "b-2"
```

13.2 11.2 Pattern matching

```
# --- Self-contained: pattern matching ---
p11_s <- c("apple", "banana", "cherry", "apricot")

grep("ap", p11_s)      # 1 4 (indices)

## [1] 1 4
grep("ap", p11_s, value = TRUE) # "apple" "apricot"

## [1] "apple" "apricot"
```

```
grepl("ap", p11_s)           # TRUE FALSE FALSE TRUE
```

```
## [1] TRUE FALSE FALSE TRUE
```

```
sub("a", "A", p11_s)         # replace first
```

```
## [1] "Apple" "bAnana" "cherry" "Apricot"
```

```
gsub("a", "A", p11_s)        # replace all
```

```
## [1] "Apple" "bAnAnA" "cherry" "Apricot"
```

13.3 11.3 stringr (tidyverse)

```
# --- Self-contained: stringr ---
```

```
p11_s <- c("apple", "banana", "cherry", "apricot")
```

```
str_detect(p11_s, "an")      # FALSE TRUE FALSE FALSE
```

```
## [1] FALSE TRUE FALSE FALSE
```

```
str_replace(p11_s, "a", "A")
```

```
## [1] "Apple" "bAnana" "cherry" "Apricot"
```

```
str_replace_all(p11_s, "a", "A")
```

```
## [1] "Apple" "bAnAnA" "cherry" "Apricot"
```

```
str_sub(p11_s, 1, 3)         # first 3 characters
```

```
## [1] "app" "ban" "che" "apr"
```

```
str_length(p11_s)
```

```
## [1] 5 6 6 7
```

```
str_to_upper(p11_s)
```

```
## [1] "APPLE" "BANANA" "CHERRY" "APRICOT"
```

```
str_trim(" hello ")
```

```
## [1] "hello"
```

```
str_pad("hi", 5, side = "left")
```

```
## [1] "   hi"
```

```
str_split("a,b,c", ",")[[1]]
```

```
## [1] "a" "b" "c"
```

13.4 11.4 Printing strings

```
# --- Self-contained: printing ---
```

```
p11_name <- "Parsa"
```

```
p11_age <- 25
```

```
cat("My name is", p11_name, "and I am", p11_age, "years old.\n")
```

```
## My name is Parsa and I am 25 years old.
print(paste("My name is", p11_name))

## [1] "My name is Parsa"
# sprintf for formatting
sprintf("Name: %s, Age: %d", p11_name, p11_age)

## [1] "Name: Parsa, Age: 25"
sprintf("Pi to 2 decimals: %.2f", pi)

## [1] "Pi to 2 decimals: 3.14"
cat for messages, print for objects, sprintf for formatted output.
```

13.5 11.5 Glue

```
# --- Self-contained: glue ---
library(glue)
p11_name <- "Parsa"
p11_age <- 25
glue("My name is {p11_name} and I am {p11_age} years old.")

## My name is Parsa and I am 25 years old.
```

14 Part 12 — Factors

14.1 12.1 Creating factors

```
# --- Self-contained: factors ---
p12_f <- factor(c("low", "high", "medium", "low", "high"))
p12_f

## [1] low    high   medium low     high
## Levels: high low medium

levels(p12_f)      # alphabetical by default

## [1] "high"  "low"   "medium"

nlevels(p12_f)

## [1] 3
```

14.2 12.2 Setting level order

```
# --- Self-contained: factor levels ---
p12_f <- factor(c("low", "high", "medium", "low", "high"),
               levels = c("low", "medium", "high"))
p12_f

## [1] low    high   medium low     high
## Levels: low medium high
```

```
levels(p12_f)
```

```
## [1] "low"      "medium" "high"
```

14.3 12.3 Changing the reference level

```
# --- Self-contained: relevel ---  
p12_f <- factor(c("A", "B", "C", "A", "B"))  
p12_f2 <- relevel(p12_f, ref = "B")  
levels(p12_f2)
```

```
## [1] "B" "A" "C"
```

14.4 12.4 Table and counts

```
# --- Self-contained: factor tables ---  
p12_f <- factor(c("low", "high", "medium", "low", "high"))  
  
table(p12_f)
```

```
## p12_f  
##   high    low medium  
##     2     2     1
```

```
prop.table(table(p12_f))
```

```
## p12_f  
##   high    low medium  
##   0.4    0.4    0.2
```

14.5 12.5 Convert to/from factor

```
# --- Self-contained: factor conversion ---  
p12_chr <- c("a", "b", "a", "c")  
p12_f <- factor(p12_chr)  
as.character(p12_f)
```

```
## [1] "a" "b" "a" "c"
```

```
as.numeric(p12_f)      # 1 2 1 3 (level codes)
```

```
## [1] 1 2 1 3
```

```
as.integer(p12_f)
```

```
## [1] 1 2 1 3
```

Warning: `as.numeric(factor)` returns level codes, not the original numbers. To get original: `as.numeric(as.character(f))`.

14.6 12.6 forcats (tidyverse)

```
# --- Self-contained: forcats ---  
library(forcats)  
p12_f <- factor(c("low", "high", "medium", "low", "high"))
```

```
fct_relevel(p12_f, "high")

## [1] low    high    medium low    high
## Levels: high low medium

fct_reorder(p12_f, c(1, 3, 2, 1, 3)) # reorder by a numeric

## [1] low    high    medium low    high
## Levels: low medium high

fct_infreq(p12_f) # most frequent first

## [1] low    high    medium low    high
## Levels: high low medium

fct_lump(p12_f, n = 2) # keep top 2, lump rest

## [1] low    high Other low    high
## Levels: high low Other
```

15 Part 13 — Dates

15.1 13.1 Creating dates

```
# --- Self-contained: dates ---
p13_d <- as.Date("2024-01-15")
p13_d

## [1] "2024-01-15"

class(p13_d)

## [1] "Date"

p13_d2 <- as.Date("15/01/2024", format = "%d/%m/%Y")
p13_d2

## [1] "2024-01-15"

p13_dt <- as.POSIXct("2024-01-15 10:30:00", tz = "UTC")
p13_dt

## [1] "2024-01-15 10:30:00 UTC"
```

15.2 13.2 Components

```
# --- Self-contained: date components ---
p13_d <- as.Date("2024-01-15")

format(p13_d, "%Y") # "2024"

## [1] "2024"

format(p13_d, "%m") # "01"

## [1] "01"
```

```

format(p13_d, "%d")    # "15"

## [1] "15"
format(p13_d, "%A")    # "Monday"

## [1] "Monday"
format(p13_d, "%B")    # "January"

## [1] "January"
weekdays(p13_d)

## [1] "Monday"
months(p13_d)

## [1] "January"
quarters(p13_d)

## [1] "Q1"

```

15.3 13.3 Arithmetic

```

# --- Self-contained: date arithmetic ---
p13_d <- as.Date("2024-01-15")

p13_d + 7                # one week later

## [1] "2024-01-22"
p13_d - 30               # 30 days earlier

## [1] "2023-12-16"
p13_d2 <- as.Date("2024-02-15")
p13_d2 - p13_d           # Time difference of 31 days

## Time difference of 31 days
as.numeric(p13_d2 - p13_d)

## [1] 31

```

15.4 13.4 Sequences

```

# --- Self-contained: date sequences ---
p13_seq <- seq(as.Date("2024-01-01"), by = "month", length.out = 6)
p13_seq

## [1] "2024-01-01" "2024-02-01" "2024-03-01" "2024-04-01" "2024-05-01"
## [6] "2024-06-01"
seq(as.Date("2024-01-01"), as.Date("2024-12-31"), by = "quarter")

## [1] "2024-01-01" "2024-04-01" "2024-07-01" "2024-10-01"

```

15.5 13.5 lubridate

```
# --- Self-contained: lubridate ---
library(lubridate)
p13_d <- ymd("2024-01-15")
p13_d

## [1] "2024-01-15"
year(p13_d)

## [1] 2024
month(p13_d)

## [1] 1
day(p13_d)

## [1] 15
wday(p13_d, label = TRUE)

## [1] Mon
## Levels: Sun < Mon < Tue < Wed < Thu < Fri < Sat
p13_d + weeks(2)

## [1] "2024-01-29"
p13_d %m+% months(1)      # month arithmetic with rollover

## [1] "2024-02-15"
```

16 Part 14 — Missing Values

16.1 14.1 Detecting

```
# --- Self-contained: missing values ---
p14_v <- c(1, NA, 3, NA, 5)

is.na(p14_v)      # FALSE TRUE FALSE TRUE FALSE
## [1] FALSE TRUE FALSE TRUE FALSE
anyNA(p14_v)      # TRUE

## [1] TRUE
sum(is.na(p14_v))  # 2

## [1] 2
mean(is.na(p14_v)) # 0.4 (fraction missing)

## [1] 0.4
complete.cases(p14_v) # TRUE FALSE TRUE FALSE TRUE
## [1] TRUE FALSE TRUE FALSE TRUE
```

16.2 14.2 Removing

```
# --- Self-contained: remove NAs ---
p14_v <- c(1, NA, 3, NA, 5)
p14_v[!is.na(p14_v)]      # 1 3 5

## [1] 1 3 5

na.omit(p14_v)             # same, with attribute

## [1] 1 3 5
## attr("na.action")
## [1] 2 4
## attr("class")
## [1] "omit"
```

16.3 14.3 Functions with na.rm

```
# --- Self-contained: na.rm ---
p14_v <- c(1, NA, 3, NA, 5)
mean(p14_v)               # NA

## [1] NA

mean(p14_v, na.rm = TRUE) # 3

## [1] 3

sum(p14_v, na.rm = TRUE)

## [1] 9

sd(p14_v, na.rm = TRUE)

## [1] 2
```

16.4 14.4 Filling / replacing

```
# --- Self-contained: fill NAs ---
p14_v <- c(1, NA, 3, NA, 5)

# Replace with zero
p14_v[is.na(p14_v)] <- 0

# Replace with mean
p14_v2 <- c(1, NA, 3, NA, 5)
p14_v2[is.na(p14_v2)] <- mean(p14_v2, na.rm = TRUE)

# Forward fill (last observation carried forward)
p14_v3 <- c(1, NA, NA, 4, NA)
p14_filled <- zoo::na.locf(p14_v3)

# Interpolation
p14_interp <- approx(seq_along(p14_v3), p14_v3, xout = seq_along(p14_v3))$y
```

16.5 14.5 NA in data frames

```
# --- Self-contained: NA in tibbles ---
p14_df <- tibble(
  x = c(1, NA, 3),
  y = c("a", "b", NA)
)
p14_df

## # A tibble: 3 x 2
##       x y
##   <dbl> <chr>
## 1     1 a
## 2    NA b
## 3     3 <NA>

tidyr::drop_na(p14_df)      # drop rows with any NA

## # A tibble: 1 x 2
##       x y
##   <dbl> <chr>
## 1     1 a

tidyr::replace_na(p14_df, list(x = 0, y = "unknown"))

## # A tibble: 3 x 2
##       x y
##   <dbl> <chr>
## 1     1 a
## 2     0 b
## 3     3 unknown
```

16.6 14.6 NA vs NULL vs NaN

Value	Meaning	In vector
NA	missing	stays in vector
NULL	empty / no value	disappears
NaN	not a number (0/0)	stays
Inf	infinity	stays

```
# --- Self-contained: NA vs NULL vs NaN ---
length(c(1, NA, 3))      # 3

## [1] 3

length(c(1, NULL, 3))    # 2

## [1] 2

length(c(1, NaN, 3))     # 3

## [1] 3
```

17 Part 15 — Reading and Writing Files

17.1 15.1 CSV

```
# --- Read ---
df <- read.csv("data.csv")
df <- read_csv("data.csv")      # tidyverse, faster
df <- read.csv("data.csv", stringsAsFactors = FALSE)

# --- Write ---
write.csv(df, "out.csv", row.names = FALSE)
write_csv(df, "out.csv")
```

17.2 15.2 Excel

```
library(readxl)
df <- read_excel("data.xlsx", sheet = 1)
```

17.3 15.3 RDS (R's native format — preserves types)

```
saveRDS(df, "data.rds")
df <- readRDS("data.rds")
```

17.4 15.4 RData (multiple objects)

```
save(df, fit, file = "workspace.RData")
load("workspace.RData")
```

17.5 15.5 Text files

```
writeLines(c("line 1", "line 2"), "out.txt")
readLines("out.txt")
```

17.6 15.6 From clipboard

```
df <- read.table("clipboard", header = TRUE) # Windows
df <- read.table(pipe("pbpaste"), header = TRUE) # macOS
```

17.7 15.7 Inspect files

```
# --- Self-contained: inspect a data frame ---
set.seed(1501)
p15_df <- tibble(x = rnorm(5), y = letters[1:5])

head(p15_df)      # first 6 rows

## # A tibble: 5 x 2
##       x y
##   <dbl> <chr>
## 1 -0.372 a
## 2  1.36  b
```

```
## 3  0.641 c
## 4 -1.27  d
## 5 -2.36  e
```

```
head(p15_df, 3)      # first 3 rows
```

```
## # A tibble: 3 x 2
##       x y
##   <dbl> <chr>
## 1 -0.372 a
## 2  1.36  b
## 3  0.641 c
```

```
tail(p15_df)
```

```
## # A tibble: 5 x 2
##       x y
##   <dbl> <chr>
## 1 -0.372 a
## 2  1.36  b
## 3  0.641 c
## 4 -1.27  d
## 5 -2.36  e
```

```
str(p15_df)          # structure
```

```
## tibble [5 x 2] (S3: tbl_df/tbl/data.frame)
##  $ x: num [1:5] -0.372 1.36 0.641 -1.274 -2.361
##  $ y: chr [1:5] "a" "b" "c" "d" ...
```

```
glimpse(p15_df)      # tidyverse structure
```

```
## Rows: 5
## Columns: 2
## $ x <dbl> -0.3715571, 1.3596495, 0.6405539, -1.2737509, -2.3606562
## $ y <chr> "a", "b", "c", "d", "e"
```

```
summary(p15_df)
```

```
##           x                y
##  Min.      :-2.3607   Length   :5
##  1st Qu.: -1.2738   N.unique  :5
##  Median :-0.3716   N.blank   :0
##  Mean    :-0.4012   Min.nchar:1
##  3rd Qu.: 0.6406   Max.nchar:1
##  Max.     : 1.3596
```

```
dim(p15_df)
```

```
## [1] 5 2
```

```
names(p15_df)
```

```
## [1] "x" "y"
```

18 Part 16 — Data Manipulation with dplyr

18.1 16.1 The five verbs

```
# --- Self-contained: dplyr verbs ---
library(dplyr)
set.seed(1601)
p16_df <- tibble(
  id    = 1:6,
  name  = c("A", "B", "C", "D", "E", "F"),
  score = c(90, 85, 88, 92, 79, 95),
  group = c("X", "X", "Y", "Y", "X", "Y")
)
```

```
# filter: keep rows
p16_df |> filter(score > 85)
```

```
## # A tibble: 4 x 4
##       id name  score group
##   <int> <chr> <dbl> <chr>
## 1     1  A      90    X
## 2     3  C      88    Y
## 3     4  D      92    Y
## 4     6  F      95    Y
```

```
# select: keep columns
p16_df |> select(name, score)
```

```
## # A tibble: 6 x 2
##   name  score
##   <chr> <dbl>
## 1 A      90
## 2 B      85
## 3 C      88
## 4 D      92
## 5 E      79
## 6 F      95
```

```
# mutate: add/modify columns
p16_df |> mutate(passed = score >= 85)
```

```
## # A tibble: 6 x 5
##       id name  score group passed
##   <int> <chr> <dbl> <chr> <lgl>
## 1     1  A      90    X     TRUE
## 2     2  B      85    X     TRUE
## 3     3  C      88    Y     TRUE
## 4     4  D      92    Y     TRUE
## 5     5  E      79    X    FALSE
## 6     6  F      95    Y     TRUE
```

```
# arrange: sort rows
p16_df |> arrange(desc(score))
```

```
## # A tibble: 6 x 4
##       id name  score group
```

```
##      <int> <chr> <dbl> <chr>
## 1         6 F          95 Y
## 2         4 D          92 Y
## 3         1 A          90 X
## 4         3 C          88 Y
## 5         2 B          85 X
## 6         5 E          79 X

# summarise: reduce to one row
p16_df |> summarise(mean_score = mean(score), n = n())

## # A tibble: 1 x 2
##   mean_score     n
##   <dbl> <int>
## 1    88.2     6
```

18.2 16.2 filter

```
# --- Self-contained: filter ---
set.seed(1602)
p16_df <- tibble(id = 1:6, score = c(90, 85, 88, 92, 79, 95),
                 group = c("X", "X", "Y", "Y", "X", "Y"))

p16_df |> filter(score > 85)
```

```
## # A tibble: 4 x 3
##       id score group
##   <int> <dbl> <chr>
## 1     1    90 X
## 2     3    88 Y
## 3     4    92 Y
## 4     6    95 Y
```

```
p16_df |> filter(score > 85 & group == "X")
```

```
## # A tibble: 1 x 3
##       id score group
##   <int> <dbl> <chr>
## 1     1    90 X
```

```
p16_df |> filter(score > 85 | group == "X")
```

```
## # A tibble: 6 x 3
##       id score group
##   <int> <dbl> <chr>
## 1     1    90 X
## 2     2    85 X
## 3     3    88 Y
## 4     4    92 Y
## 5     5    79 X
## 6     6    95 Y
```

```
p16_df |> filter(group %in% c("X", "Y"))
```

```
## # A tibble: 6 x 3
##       id score group
##   <int> <dbl> <chr>
```

```
## 1      1      90 X
## 2      2      85 X
## 3      3      88 Y
## 4      4      92 Y
## 5      5      79 X
## 6      6      95 Y
```

```
p16_df |> filter(!is.na(score))
```

```
## # A tibble: 6 x 3
##       id score group
##   <int> <dbl> <chr>
## 1     1     90 X
## 2     2     85 X
## 3     3     88 Y
## 4     4     92 Y
## 5     5     79 X
## 6     6     95 Y
```

18.3 16.3 select

```
# --- Self-contained: select ---
set.seed(1603)
p16_df <- tibble(id = 1:3, name = c("A","B","C"),
                 score = c(90,85,88), group = c("X","X","Y"))
```

```
p16_df |> select(name, score)
```

```
## # A tibble: 3 x 2
##   name score
##   <chr> <dbl>
## 1 A      90
## 2 B      85
## 3 C      88
```

```
p16_df |> select(-id) # drop
```

```
## # A tibble: 3 x 3
##   name score group
##   <chr> <dbl> <chr>
## 1 A      90 X
## 2 B      85 X
## 3 C      88 Y
```

```
p16_df |> select(starts_with("s"))
```

```
## # A tibble: 3 x 1
##   score
##   <dbl>
## 1     90
## 2     85
## 3     88
```

```
p16_df |> select(where(is.numeric))
```

```
## # A tibble: 3 x 2
##       id score
```

```
##   <int> <dbl>
## 1     1     90
## 2     2     85
## 3     3     88
```

```
p16_df |> select(name:score)           # range
```

```
## # A tibble: 3 x 2
##   name  score
##   <chr> <dbl>
## 1 A      90
## 2 B      85
## 3 C      88
```

```
p16_df |> relocate(score, .before = name)
```

```
## # A tibble: 3 x 4
##       id score name  group
##   <int> <dbl> <chr> <chr>
## 1     1     90 A      X
## 2     2     85 B      X
## 3     3     88 C      Y
```

18.4 16.4 mutate

```
# --- Self-contained: mutate ---
set.seed(1604)
p16_df <- tibble(x = c(1, 2, 3), y = c(4, 5, 6))

p16_df |> mutate(z = x + y)
```

```
## # A tibble: 3 x 3
##       x     y     z
##   <dbl> <dbl> <dbl>
## 1     1     4     5
## 2     2     5     7
## 3     3     6     9
```

```
p16_df |> mutate(z = x + y, w = z * 2)           # uses z just created
```

```
## # A tibble: 3 x 4
##       x     y     z     w
##   <dbl> <dbl> <dbl> <dbl>
## 1     1     4     5    10
## 2     2     5     7    14
## 3     3     6     9    18
```

```
p16_df |> mutate(x2 = x^2, x_cat = if_else(x > 1, "big", "small"))
```

```
## # A tibble: 3 x 4
##       x     y    x2 x_cat
##   <dbl> <dbl> <dbl> <chr>
## 1     1     4     1 small
## 2     2     5     4 big
## 3     3     6     9 big
```

```
p16_df |> transmute(z = x + y) # keep only new cols
```

```
## # A tibble: 3 x 1
##       z
##   <dbl>
## 1     5
## 2     7
## 3     9
```

18.5 16.5 arrange

```
# --- Self-contained: arrange ---
set.seed(1605)
p16_df <- tibble(name = c("A","B","C","D"), score = c(90,85,88,92),
                 group = c("X","X","Y","Y"))

p16_df |> arrange(score)
```

```
## # A tibble: 4 x 3
##   name  score group
##   <chr> <dbl> <chr>
## 1 B      85 X
## 2 C      88 Y
## 3 A      90 X
## 4 D      92 Y
```

```
p16_df |> arrange(desc(score))
```

```
## # A tibble: 4 x 3
##   name  score group
##   <chr> <dbl> <chr>
## 1 D      92 Y
## 2 A      90 X
## 3 C      88 Y
## 4 B      85 X
```

```
p16_df |> arrange(group, desc(score))
```

```
## # A tibble: 4 x 3
##   name  score group
##   <chr> <dbl> <chr>
## 1 A      90 X
## 2 B      85 X
## 3 D      92 Y
## 4 C      88 Y
```

18.6 16.6 summarise + group_by

```
# --- Self-contained: group_by + summarise ---
set.seed(1606)
p16_df <- tibble(group = c("X","X","Y","Y","X","Y"),
                 score = c(90,85,88,92,79,95))

p16_df |>
```

```
group_by(group) |>
summarise(mean_score = mean(score), n = n(), .groups = "drop")
```

```
## # A tibble: 2 x 3
##   group mean_score     n
##   <chr>     <dbl> <int>
## 1 X         84.7     3
## 2 Y         91.7     3
```

Multiple summaries

```
p16_df |>
group_by(group) |>
summarise(
  mean = mean(score),
  sd    = sd(score),
  min   = min(score),
  max   = max(score),
  n     = n(),
  .groups = "drop"
)
```

```
## # A tibble: 2 x 6
##   group mean    sd  min  max    n
##   <chr> <dbl> <dbl> <dbl> <dbl> <int>
## 1 X     84.7  5.51   79   90     3
## 2 Y     91.7  3.51   88   95     3
```

18.7 16.7 count and tally

```
# --- Self-contained: count ---
set.seed(1607)
p16_df <- tibble(group = c("X", "X", "Y", "Y", "X", "Y"))

p16_df |> count(group)
```

```
## # A tibble: 2 x 2
##   group     n
##   <chr> <int>
## 1 X         3
## 2 Y         3
```

```
p16_df |> count(group, sort = TRUE)
```

```
## # A tibble: 2 x 2
##   group     n
##   <chr> <int>
## 1 X         3
## 2 Y         3
```

18.8 16.8 slice and distinct

```
# --- Self-contained: slice and distinct ---
set.seed(1608)
p16_df <- tibble(x = c(1, 2, 2, 3, 3, 3))
```

```
p16_df |> distinct()
```

```
## # A tibble: 3 x 1
##       x
##   <dbl>
## 1     1
## 2     2
## 3     3
```

```
p16_df |> distinct(x)
```

```
## # A tibble: 3 x 1
##       x
##   <dbl>
## 1     1
## 2     2
## 3     3
```

```
p16_df |> slice(1:2)
```

```
## # A tibble: 2 x 1
##       x
##   <dbl>
## 1     1
## 2     2
```

```
p16_df |> slice_head(n = 2)
```

```
## # A tibble: 2 x 1
##       x
##   <dbl>
## 1     1
## 2     2
```

```
p16_df |> slice_max(x, n = 2)
```

```
## # A tibble: 3 x 1
##       x
##   <dbl>
## 1     3
## 2     3
## 3     3
```

```
p16_df |> slice_min(x, n = 2)
```

```
## # A tibble: 3 x 1
##       x
##   <dbl>
## 1     1
## 2     2
## 3     2
```

```
p16_df |> slice_sample(n = 2)
```

```
## # A tibble: 2 x 1
##       x
##   <dbl>
## 1     3
```

```
## 2      2
```

18.9 16.9 pull and rename

```
# --- Self-contained: pull and rename ---
set.seed(1609)
p16_df <- tibble(id = 1:3, score = c(90, 85, 88))

p16_df |> pull(score)          # extract a column as vector

## [1] 90 85 88

p16_df |> rename(points = score) # rename columns
```

```
## # A tibble: 3 x 2
##       id points
##   <int> <dbl>
## 1     1     90
## 2     2     85
## 3     3     88
```

18.10 16.10 across (apply to many columns)

```
# --- Self-contained: across ---
set.seed(1610)
p16_df <- tibble(a = rnorm(10), b = rnorm(10), g = rep(c("X", "Y"), 5))

p16_df |>
  group_by(g) |>
  summarise(across(where(is.numeric), mean), .groups = "drop")

## # A tibble: 2 x 3
##       g         a         b
##   <chr> <dbl> <dbl>
## 1 X     0.178 -0.434
## 2 Y     0.736  0.688

p16_df |> mutate(across(where(is.numeric), ~ .x * 2))
```

```
## # A tibble: 10 x 3
##       a         b g
##   <dbl> <dbl> <chr>
## 1  1.29 -0.983 X
## 2  2.24  3.25  Y
## 3  2.44 -2.46  X
## 4  0.148 0.411 Y
## 5 -0.603 0.288 X
## 6  0.928 2.30  Y
## 7  1.57 -2.94  X
## 8  3.13 -0.776 Y
## 9 -2.90  1.76  X
## 10 0.917 1.69  Y
```

18.11 16.11 Summary of dplyr verbs

Verb	Purpose
<code>filter()</code>	keep rows matching condition
<code>select()</code>	keep columns
<code>mutate()</code>	add / modify columns
<code>arrange()</code>	sort rows
<code>summarise()</code>	reduce to summary rows
<code>group_by()</code>	group for summary
<code>count()</code>	count rows by group
<code>distinct()</code>	unique rows
<code>slice()</code>	pick rows by position
<code>rename()</code>	rename columns
<code>relocate()</code>	move columns
<code>pull()</code>	extract a column as vector
<code>across()</code>	apply function to many columns

19 Part 17 — Reshaping with tidyr

19.1 17.1 `pivot_longer` (wide → long)

```
# --- Self-contained: pivot_longer ---
set.seed(1701)
p17_wide <- tibble(
  id = 1:3,
  q1 = c(10, 11, 12),
  q2 = c(20, 21, 22),
  q3 = c(30, 31, 32)
)
p17_wide
```

```
## # A tibble: 3 x 4
##   id    q1    q2    q3
##   <int> <dbl> <dbl> <dbl>
## 1     1    10    20    30
## 2     2    11    21    31
## 3     3    12    22    32
```

```
p17_long <- p17_wide |>
  pivot_longer(cols = starts_with("q"),
    names_to = "quarter",
    values_to = "value")
p17_long
```

```
## # A tibble: 9 x 3
##   id quarter value
##   <int> <chr>   <dbl>
## 1     1 q1       10
## 2     1 q2       20
## 3     1 q3       30
## 4     2 q1       11
## 5     2 q2       21
## 6     2 q3       31
```

```
## 7      3 q1      12
## 8      3 q2      22
## 9      3 q3      32
```

19.2 17.2 pivot_wider (long → wide)

```
# --- Self-contained: pivot_wider ---
p17_long <- tibble(
  id      = rep(1:3, each = 3),
  quarter = rep(c("q1", "q2", "q3"), 3),
  value   = c(10, 20, 30, 11, 21, 31, 12, 22, 32)
)
p17_long

## # A tibble: 9 x 3
##       id quarter value
##   <int> <chr>   <dbl>
## 1     1   q1      10
## 2     1   q2      20
## 3     1   q3      30
## 4     2   q1      11
## 5     2   q2      21
## 6     2   q3      31
## 7     3   q1      12
## 8     3   q2      22
## 9     3   q3      32

p17_wide <- p17_long |>
  pivot_wider(names_from = quarter, values_from = value)
p17_wide

## # A tibble: 3 x 4
##       id    q1    q2    q3
##   <int> <dbl> <dbl> <dbl>
## 1     1    10    20    30
## 2     2    11    21    31
## 3     3    12    22    32
```

19.3 17.3 separate and unite

```
# --- Self-contained: separate and unite ---
p17_df <- tibble(full = c("A-1", "B-2", "C-3"))

p17_df |> separate(full, into = c("letter", "number"), sep = "-")

## # A tibble: 3 x 2
##   letter number
##   <chr>   <chr>
## 1 A      1
## 2 B      2
## 3 C      3

p17_df2 <- tibble(letter = c("A", "B"), number = c(1, 2))
p17_df2 |> unite("full", letter, number, sep = "-")
```

```
## # A tibble: 2 x 1
##   full
##   <chr>
## 1 A-1
## 2 B-2
```

19.4 17.4 Complete and fill

```
# --- Self-contained: complete ---
p17_df <- tibble(
  year = c(2020, 2021, 2023),
  val  = c(1, 2, 4)
)
p17_df |> tidyr::complete(year = 2020:2023)
```

```
## # A tibble: 4 x 2
##   year  val
##   <dbl> <dbl>
## 1 2020    1
## 2 2021    2
## 3 2022   NA
## 4 2023    4
```

```
# Fill missing values from previous row
p17_df2 <- tibble(x = c(1, NA, NA, 4, NA))
p17_df2 |> fill(x, .direction = "down")
```

```
## # A tibble: 5 x 1
##       x
##   <dbl>
## 1     1
## 2     1
## 3     1
## 4     4
## 5     4
```

20 Part 18 — Joins

20.1 18.1 The four main joins

```
# --- Self-contained: joins ---
p18_a <- tibble(id = c(1, 2, 3), x = c("a", "b", "c"))
p18_b <- tibble(id = c(2, 3, 4), y = c("B", "C", "D"))

# Inner: only matching rows
p18_a |> inner_join(p18_b, by = "id")
```

```
## # A tibble: 2 x 3
##       id x     y
##   <dbl> <chr> <chr>
## 1     2 b     B
## 2     3 c     C
```

```
# Left: all rows in a
p18_a |> left_join(p18_b, by = "id")
```

```
## # A tibble: 3 x 3
##       id x     y
##   <dbl> <chr> <chr>
## 1     1 a    <NA>
## 2     2 b     B
## 3     3 c     C
```

```
# Right: all rows in b
p18_a |> right_join(p18_b, by = "id")
```

```
## # A tibble: 3 x 3
##       id x     y
##   <dbl> <chr> <chr>
## 1     2 b     B
## 2     3 c     C
## 3     4 <NA> D
```

```
# Full: all rows in both
p18_a |> full_join(p18_b, by = "id")
```

```
## # A tibble: 4 x 3
##       id x     y
##   <dbl> <chr> <chr>
## 1     1 a    <NA>
## 2     2 b     B
## 3     3 c     C
## 4     4 <NA> D
```

20.2 18.2 Anti and semi joins

```
# --- Self-contained: anti and semi ---
p18_a <- tibble(id = c(1, 2, 3), x = c("a","b","c"))
p18_b <- tibble(id = c(2, 3, 4), y = c("B","C","D"))
```

```
# Anti: rows in a NOT in b
p18_a |> anti_join(p18_b, by = "id")
```

```
## # A tibble: 1 x 2
##       id x
##   <dbl> <chr>
## 1     1 a
```

```
# Semi: rows in a that ARE in b (only a's columns)
p18_a |> semi_join(p18_b, by = "id")
```

```
## # A tibble: 2 x 2
##       id x
##   <dbl> <chr>
## 1     2 b
## 2     3 c
```

20.3 18.3 Joining by different names

```
# --- Self-contained: join with different key names ---
p18_a <- tibble(id_a = c(1, 2, 3), x = c("a","b","c"))
p18_b <- tibble(id_b = c(2, 3, 4), y = c("B","C","D"))

p18_a |> inner_join(p18_b, by = c("id_a" = "id_b"))
```

```
## # A tibble: 2 x 3
##   id_a x      y
##   <dbl> <chr> <chr>
## 1     2 b      B
## 2     3 c      C
```

20.4 18.4 Join summary

Join	Keeps
inner_join()	matches only
left_join()	all rows from left
right_join()	all rows from right
full_join()	all rows from both
anti_join()	rows in left NOT in right
semi_join()	rows in left that match right

21 Part 19 — purrr (Functional Programming)

21.1 19.1 map

```
# --- Self-contained: map ---
library(purrr)
p19_list <- list(a = 1:3, b = 4:6, c = 7:9)

map(p19_list, sum)          # list

## $a
## [1] 6
##
## $b
## [1] 15
##
## $c
## [1] 24

map_dbl(p19_list, sum)      # numeric vector

## a b c
## 6 15 24

map_chr(p19_list, ~ paste(.x, collapse = "-"))

## a b c
## "1-2-3" "4-5-6" "7-8-9"
```

```
map_lgl(p19_list, ~ any(.x > 5))
```

```
##      a      b      c  
## FALSE TRUE  TRUE
```

21.2 19.2 map over data frame columns

```
# --- Self-contained: map over columns ---  
set.seed(1902)  
p19_df <- tibble(a = rnorm(10), b = rnorm(10), c = rnorm(10))  
  
map_dbl(p19_df, mean)
```

```
##      a      b      c  
## 0.19752556 0.10518637 0.01279868
```

```
map_dbl(p19_df, sd)
```

```
##      a      b      c  
## 1.4518403 1.0145010 0.8549233
```

21.3 19.3 map2 and pmap

```
# --- Self-contained: map2 and pmap ---  
map2_dbl(1:3, 4:6, ~ .x + .y)
```

```
## [1] 5 7 9
```

```
pmap_dbl(list(1:3, 4:6, 7:9), ~ ..1 + ..2 + ..3)
```

```
## [1] 12 15 18
```

21.4 19.4 walk (for side effects)

```
# --- Self-contained: walk ---  
p19_list <- list(1, 2, 3)  
walk(p19_list, ~ cat("Value:", .x, "\n"))
```

```
## Value: 1  
## Value: 2  
## Value: 3
```

22 Part 20 — Base R Plots

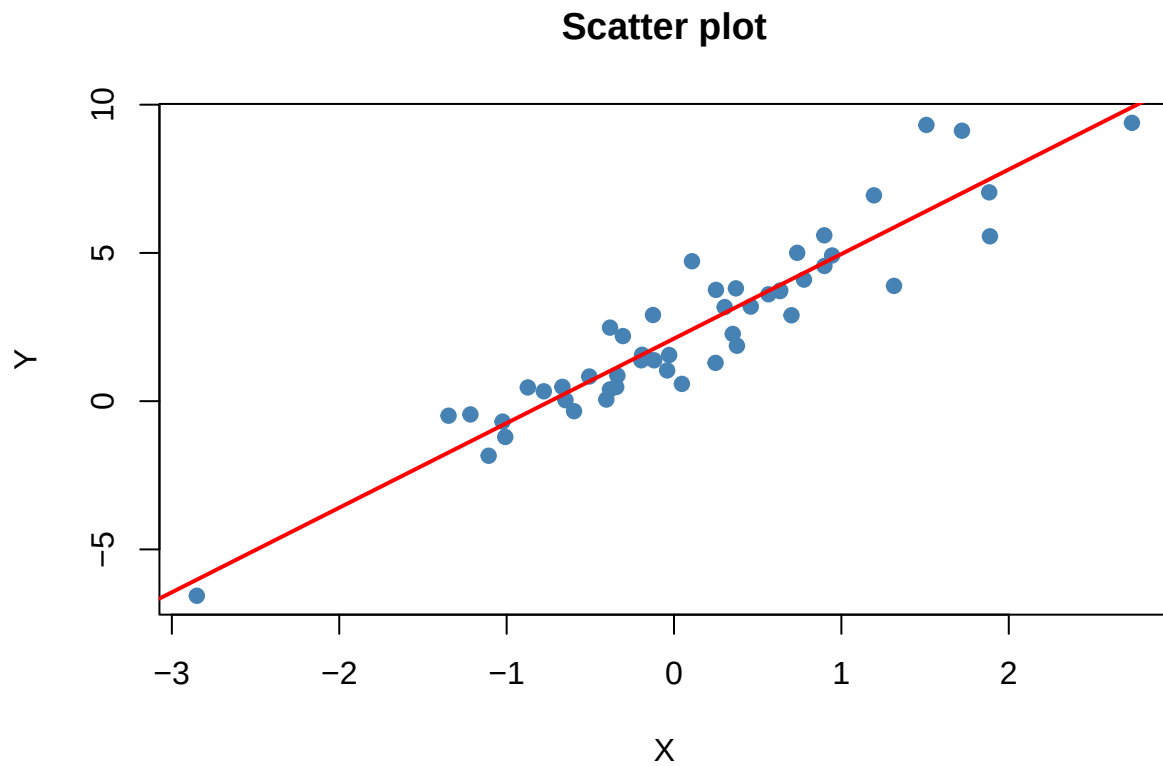
22.1 20.1 Scatter plot

```
# --- Self-contained: base scatter ---  
set.seed(2001)  
p20_x <- rnorm(50)  
p20_y <- 2 + 3 * p20_x + rnorm(50)  
  
plot(p20_x, p20_y,  
     main = "Scatter plot",
```

```

xlab = "X", ylab = "Y",
col = "steelblue", pch = 19)
abline(lm(p20_y ~ p20_x), col = "red", lwd = 2)

```



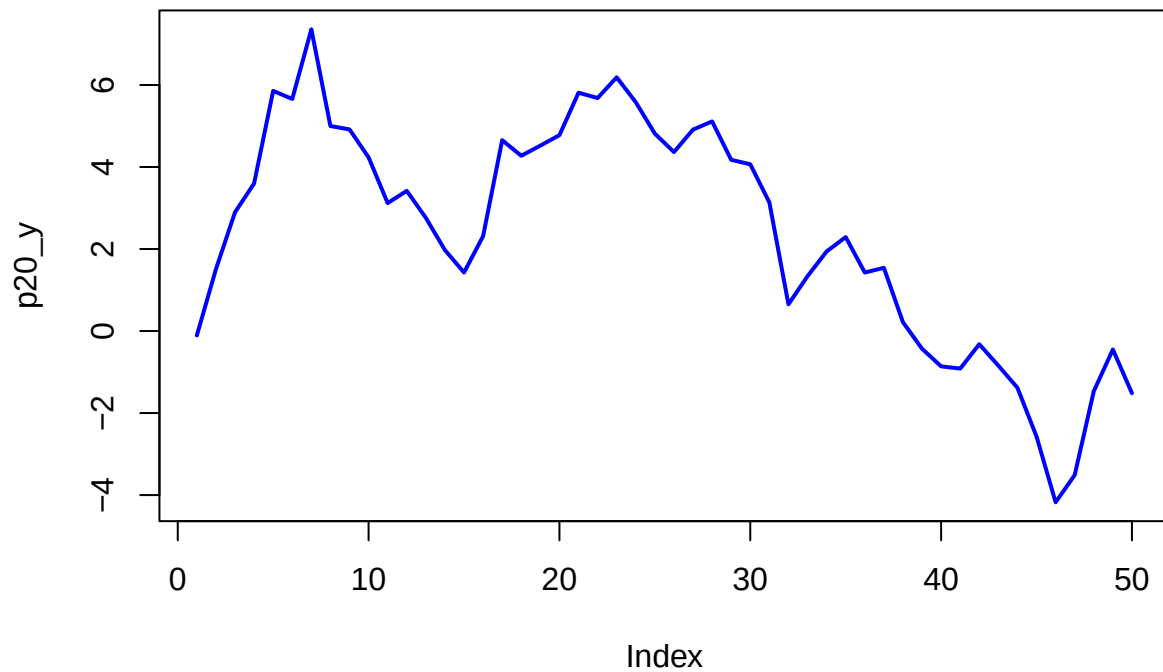
22.2 20.2 Line plot

```

# --- Self-contained: base line plot ---
set.seed(2002)
p20_y <- cumsum(rnorm(50))
plot(p20_y, type = "l", main = "Line plot", col = "blue", lwd = 2)

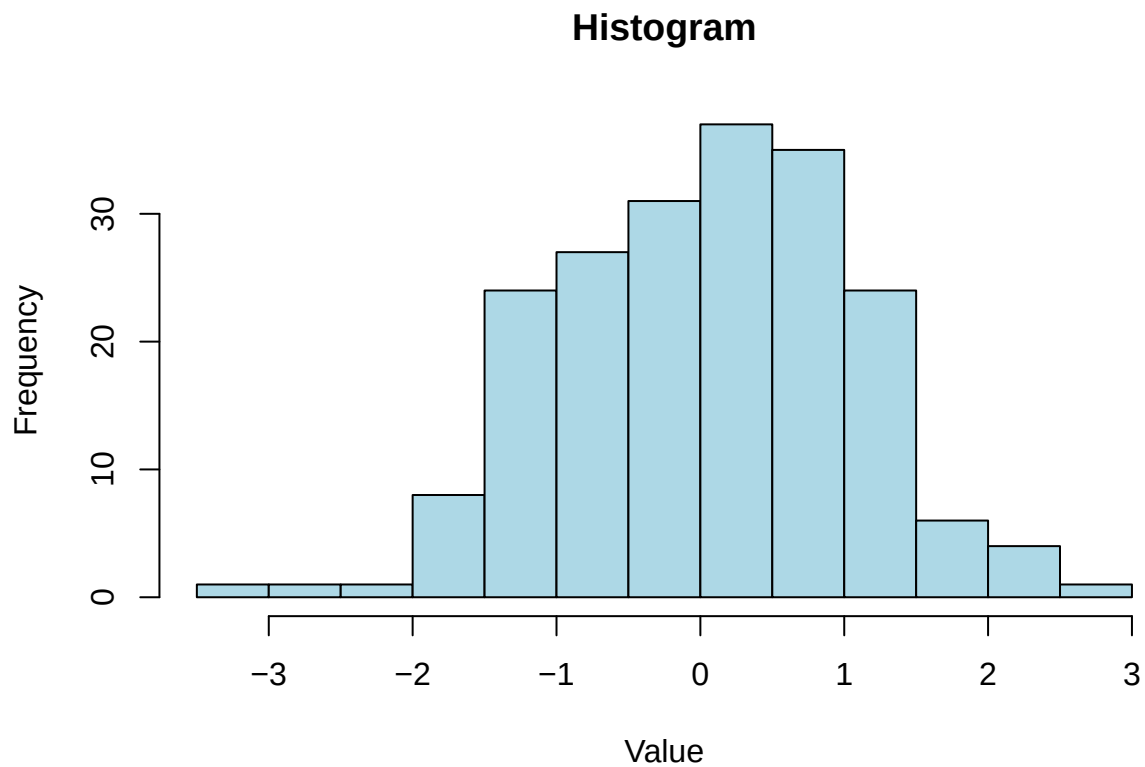
```

Line plot



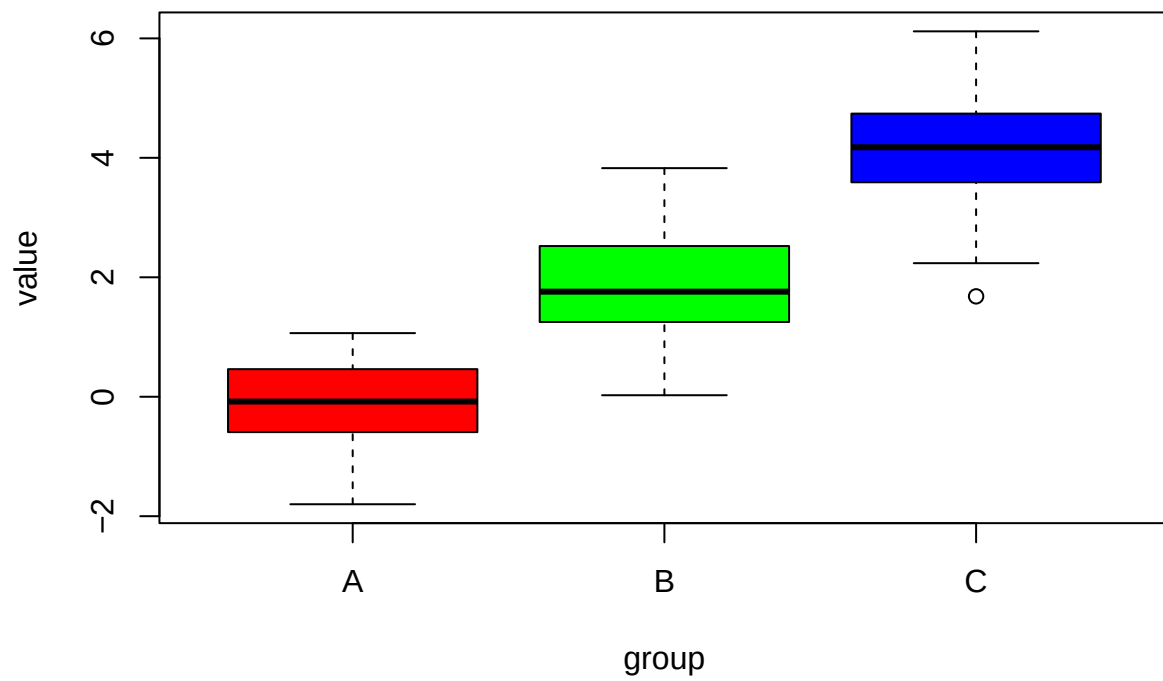
22.3 20.3 Histogram

```
# --- Self-contained: histogram ---  
set.seed(2003)  
p20_v <- rnorm(200)  
hist(p20_v, breaks = 20, col = "lightblue",  
     main = "Histogram", xlab = "Value")
```



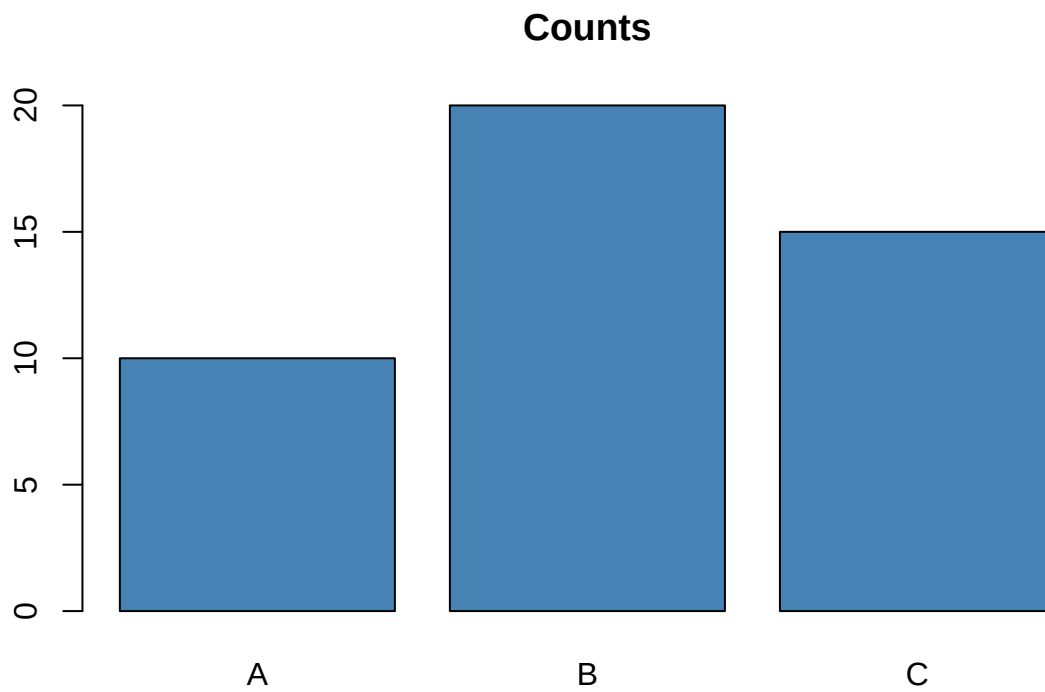
22.4 20.4 Boxplot

```
# --- Self-contained: boxplot ---  
set.seed(2004)  
p20_df <- data.frame(  
  group = rep(c("A", "B", "C"), each = 20),  
  value = c(rnorm(20), rnorm(20, 2), rnorm(20, 4))  
)  
boxplot(value ~ group, data = p20_df, col = c("red", "green", "blue"))
```



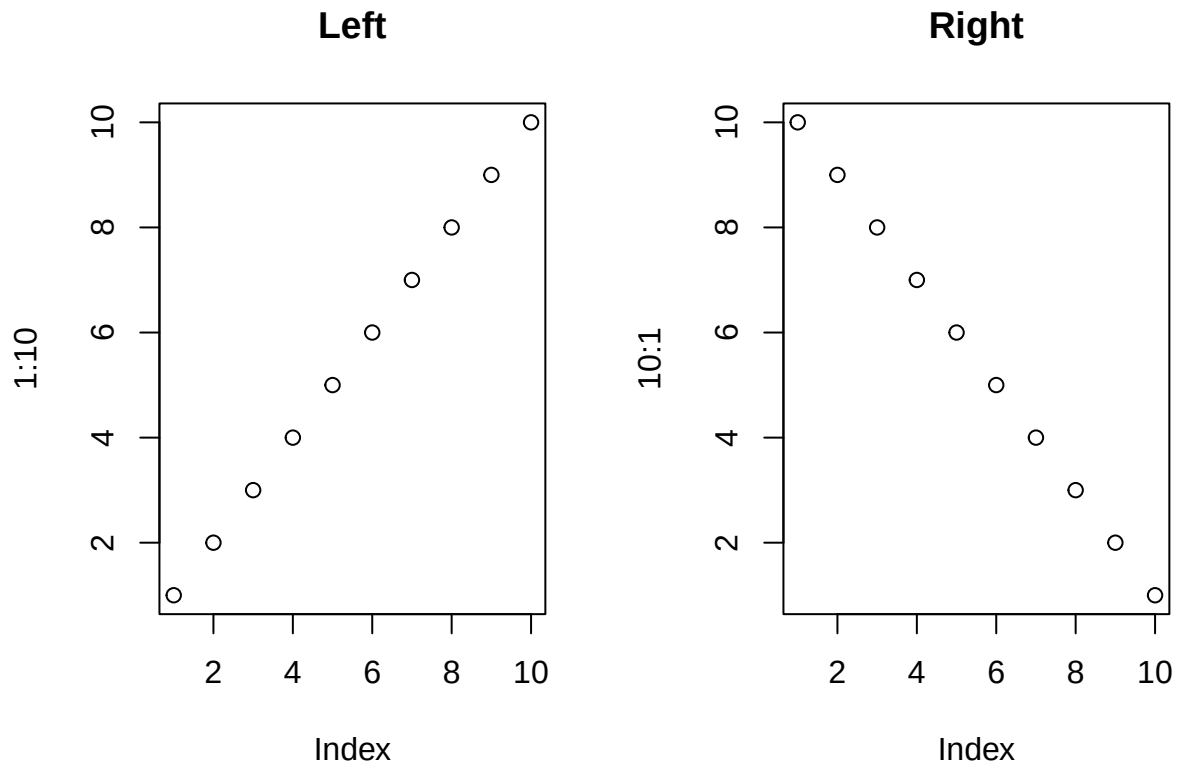
22.5 20.5 Bar plot

```
# --- Self-contained: barplot ---  
p20_counts <- c(A = 10, B = 20, C = 15)  
barplot(p20_counts, col = "steelblue", main = "Counts")
```



22.6 20.6 Layout: multiple plots

```
# --- Self-contained: multiple plots ---  
par(mfrow = c(1, 2))      # 1 row, 2 cols  
plot(1:10, main = "Left")  
plot(10:1, main = "Right")
```



```
par(mfrow = c(1, 1))      # reset
```

22.7 20.7 Save a plot

```
png("plot.png", width = 800, height = 600)
plot(1:10)
dev.off()

pdf("plot.pdf", width = 8, height = 6)
plot(1:10)
dev.off()
```

23 Part 21 — ggplot2

23.1 21.1 Grammar of graphics

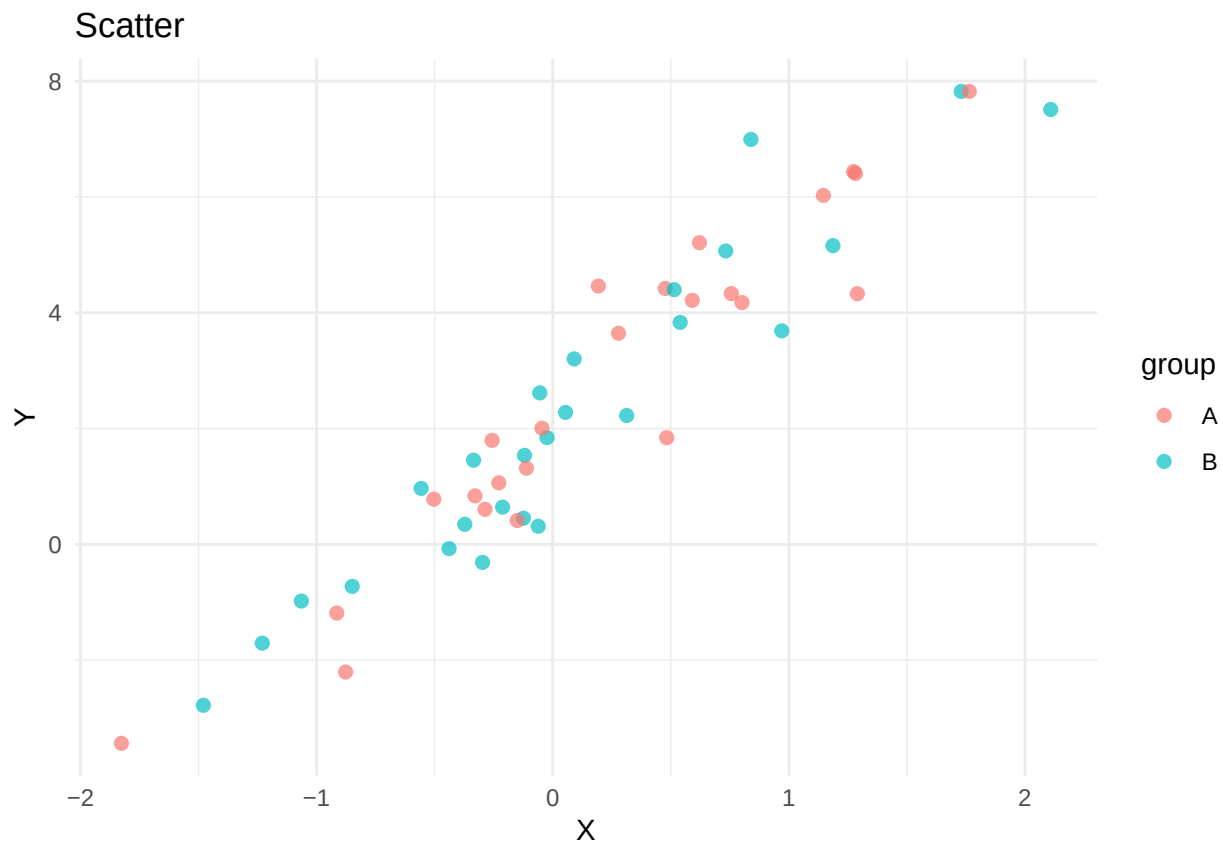
Every ggplot has:

1. **Data:** `ggplot(data = df)`
2. **Aesthetics:** `aes(x = x, y = y, color = group)`
3. **Geometries:** `+ geom_point()`, `+ geom_line()`, etc.
4. **Facets:** `+ facet_wrap(~ group)`
5. **Scales:** `+ scale_color_manual(...)`
6. **Theme:** `+ theme_minimal()`

23.2 21.2 Scatter plot

```
# --- Self-contained: ggplot scatter ---
set.seed(2101)
p21_df <- tibble(x = rnorm(50), y = 2 + 3 * x + rnorm(50),
                 group = sample(c("A", "B"), 50, replace = TRUE))

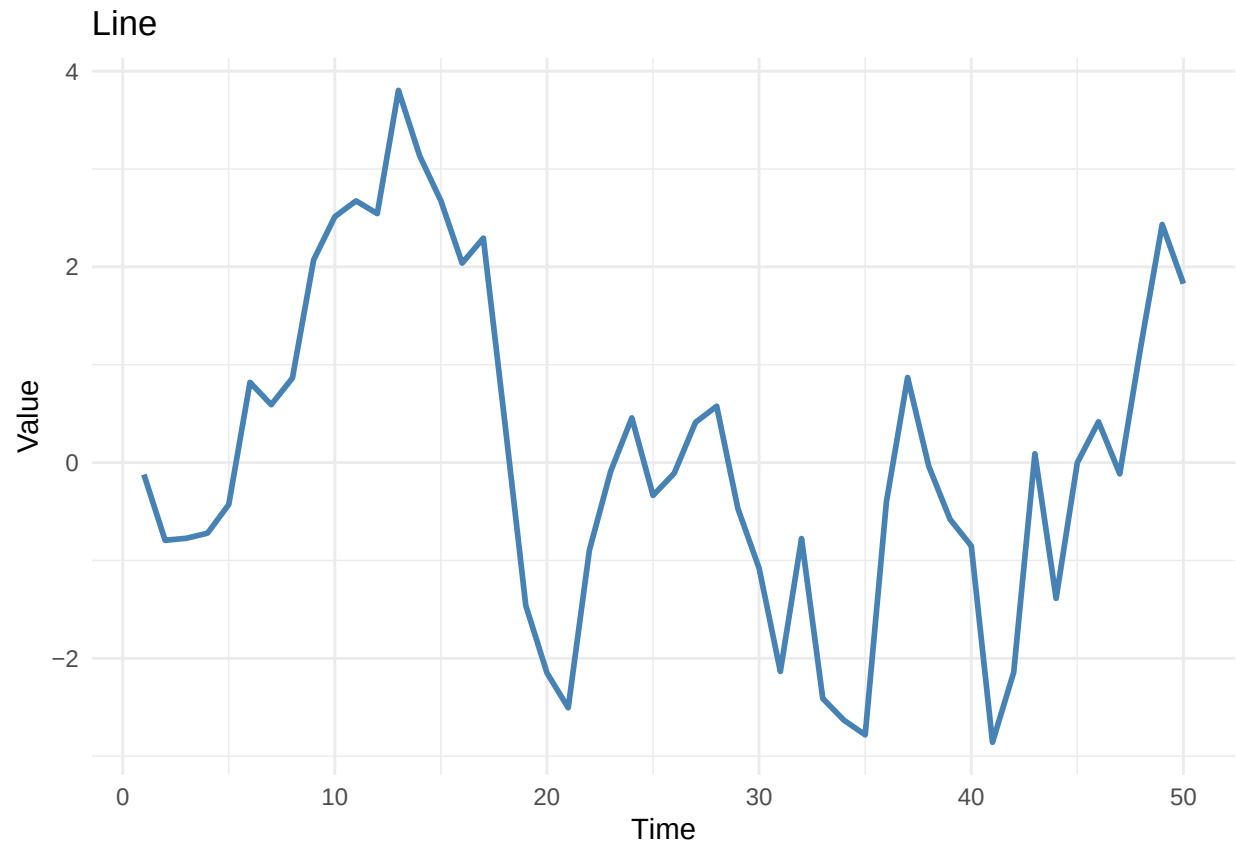
ggplot(p21_df, aes(x = x, y = y, color = group)) +
  geom_point(size = 2, alpha = 0.7) +
  labs(title = "Scatter", x = "X", y = "Y") +
  theme_minimal()
```



23.3 21.3 Line plot

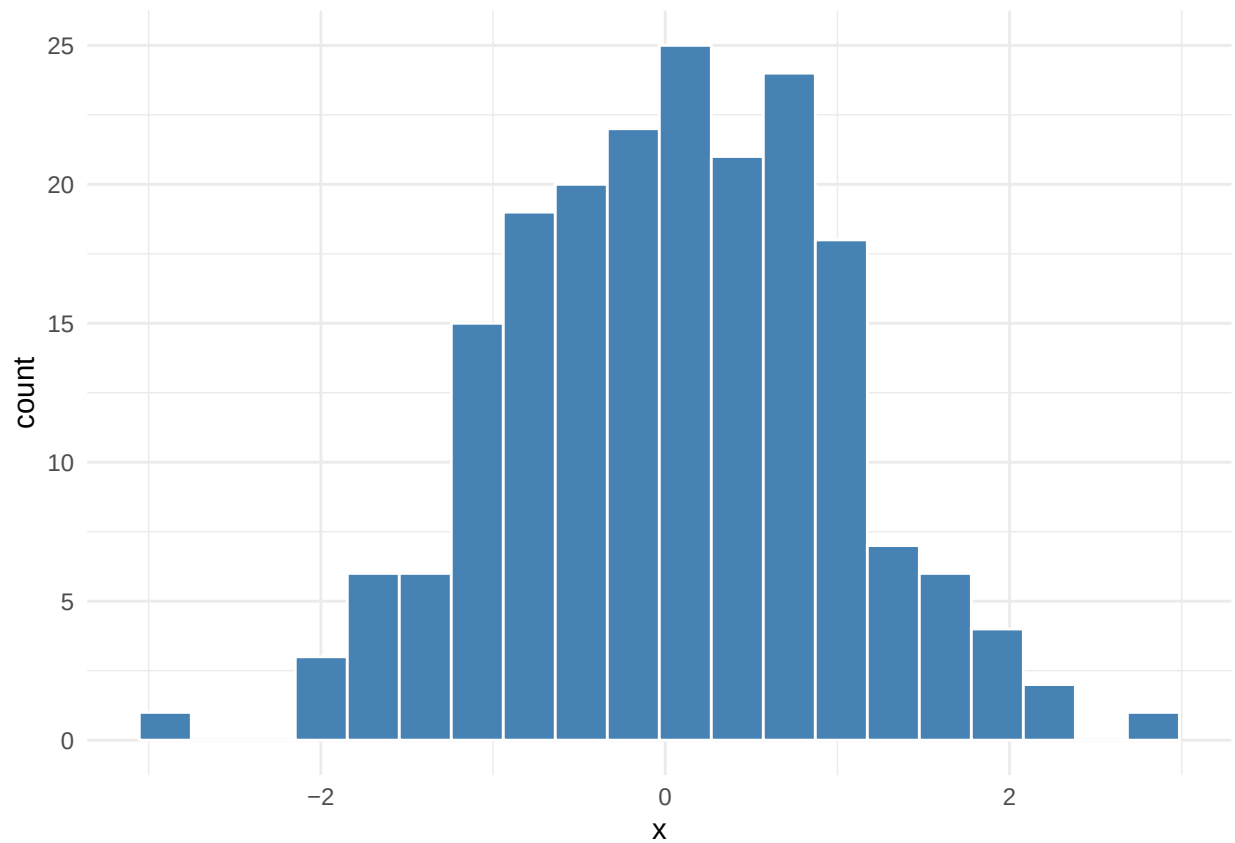
```
# --- Self-contained: ggplot line ---
set.seed(2102)
p21_df <- tibble(t = 1:50, y = cumsum(rnorm(50)))

ggplot(p21_df, aes(t, y)) +
  geom_line(color = "steelblue", linewidth = 1) +
  labs(title = "Line", x = "Time", y = "Value") +
  theme_minimal()
```



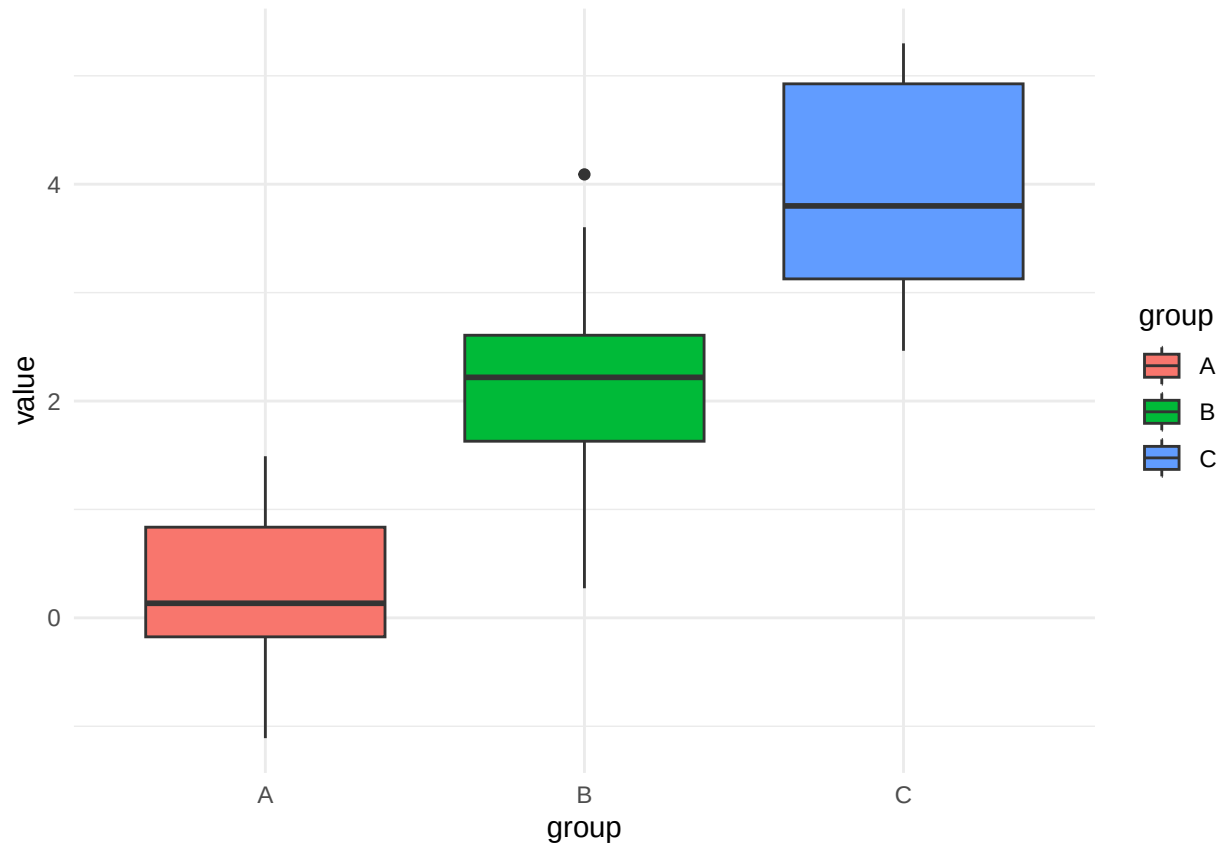
23.4 21.4 Histogram

```
# --- Self-contained: ggplot histogram ---  
set.seed(2103)  
p21_df <- tibble(x = rnorm(200))  
  
ggplot(p21_df, aes(x)) +  
  geom_histogram(bins = 20, fill = "steelblue", color = "white") +  
  theme_minimal()
```



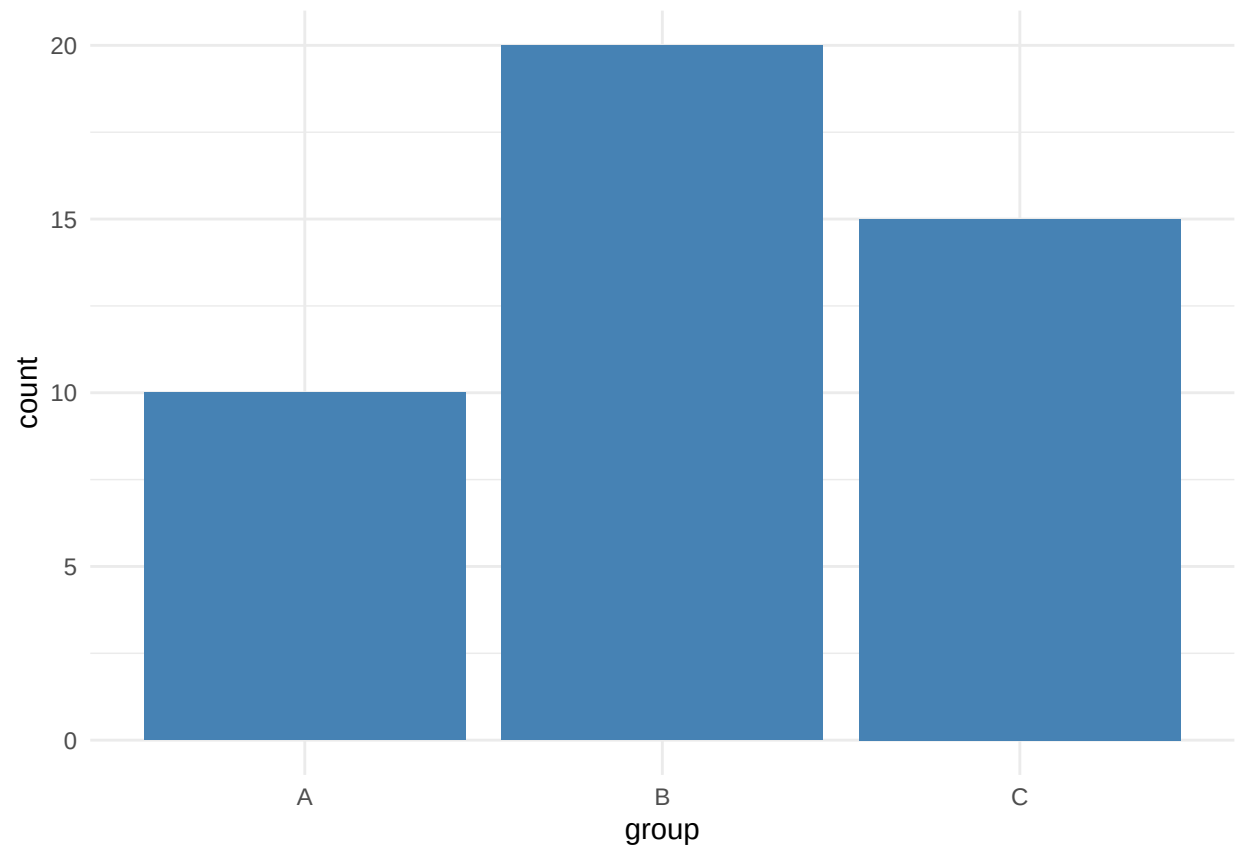
23.5 21.5 Boxplot

```
# --- Self-contained: ggplot boxplot ---  
set.seed(2104)  
p21_df <- tibble(  
  group = rep(c("A", "B", "C"), each = 20),  
  value = c(rnorm(20), rnorm(20, 2), rnorm(20, 4))  
)  
  
ggplot(p21_df, aes(group, value, fill = group)) +  
  geom_boxplot() +  
  theme_minimal()
```



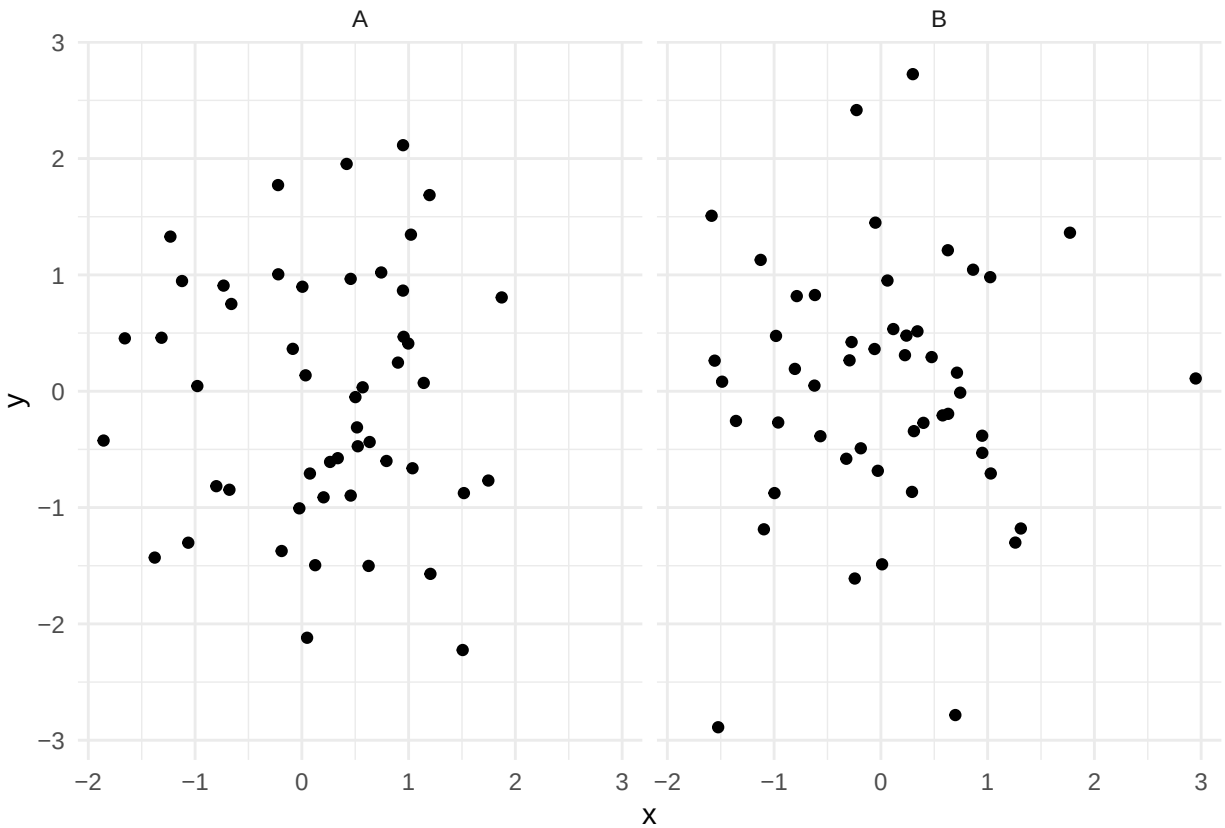
23.6 21.6 Bar plot

```
# --- Self-contained: ggplot bar ---  
p21_df <- tibble(group = c("A", "B", "C"), count = c(10, 20, 15))  
  
ggplot(p21_df, aes(group, count)) +  
  geom_col(fill = "steelblue") +  
  theme_minimal()
```



23.7 21.7 Facets

```
# --- Self-contained: facets ---  
set.seed(2107)  
p21_df <- tibble(  
  x = rnorm(100),  
  y = rnorm(100),  
  group = rep(c("A", "B"), 50)  
)  
  
ggplot(p21_df, aes(x, y)) +  
  geom_point() +  
  facet_wrap(~ group) +  
  theme_minimal()
```



23.8 21.8 Saving

```
ggsave("plot.png", width = 6, height = 4, dpi = 300)
ggsave("plot.pdf", width = 6, height = 4)
```

23.9 21.9 Common geoms

Geom	Use for
<code>geom_point()</code>	scatter
<code>geom_line()</code>	line
<code>geom_bar()</code> / <code>geom_col()</code>	bar
<code>geom_histogram()</code>	histogram
<code>geom_boxplot()</code>	boxplot
<code>geom_smooth(method = "lm")</code>	regression line
<code>geom_text()</code>	labels
<code>geom_hline(yintercept = ...)</code>	horizontal line
<code>geom_vline(xintercept = ...)</code>	vertical line
<code>geom_ribbon()</code>	confidence bands

23.10 21.10 Common theme tweaks

```
+ theme_minimal()
+ theme_bw()
```

```
+ theme_classic()
+ labs(title = "...", x = "...", y = "...")
+ scale_color_manual(values = c("red", "blue"))
+ scale_x_continuous(limits = c(0, 10))
+ coord_flip()
```

24 Part 22 — Statistical Functions

24.1 22.1 Descriptive

```
# --- Self-contained: descriptive stats ---
set.seed(2201)
p22_x <- rnorm(100)

mean(p22_x)

## [1] -0.01598462
median(p22_x)

## [1] -0.02116783
sd(p22_x)

## [1] 0.8841237
var(p22_x)

## [1] 0.7816747
range(p22_x)

## [1] -1.960894 2.514329
quantile(p22_x, c(0.025, 0.5, 0.975))

##          2.5%          50%          97.5%
## -1.57088149 -0.02116783  1.58814404
IQR(p22_x)

## [1] 0.9874762
summary(p22_x)

##      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
## -1.96089 -0.52011 -0.02117 -0.01598  0.46737  2.51433
```

24.2 22.2 Distributions

For each distribution, R has 4 functions: **d** (density), **p** (CDF), **q** (quantile), **r** (random).

Distribution	Functions
Normal	<code>dnorm</code> , <code>pnorm</code> , <code>qnorm</code> , <code>rnorm</code>
Binomial	<code>dbinom</code> , <code>pbinom</code> , <code>qbinom</code> , <code>rbinom</code>
Poisson	<code>dpois</code> , <code>ppois</code> , <code>qpois</code> , <code>rpois</code>

Distribution	Functions
Uniform	dunif, punif, qunif, runif
Exponential	dexp, pexp, qexp, rexp
t	dt, pt, qt, rt
Chi-square	dchisq, pchisq, qchisq, rchisq
F	df, pf, qf, rf

```
# --- Self-contained: distribution functions ---
# Density at  $x = 0$  for  $N(0, 1)$ 
dnorm(0)

## [1] 0.3989423
# CDF at  $x = 1.96$ 
pnorm(1.96)

## [1] 0.9750021
# 97.5% quantile of  $N(0, 1)$ 
qnorm(0.975)

## [1] 1.959964
# 5 random draws from  $N(0, 1)$ 
set.seed(2202)
rnorm(5)

## [1] -0.1238476 -0.6405983 -0.7612881 -0.2602927 1.3553755
```

24.3 22.3 Hypothesis tests

```
# --- Self-contained: hypothesis tests ---
set.seed(2203)
p22_x <- rnorm(30)

# One-sample t-test
t.test(p22_x, mu = 0)

##
## One Sample t-test
##
## data:  p22_x
## t = -1.5184, df = 29, p-value = 0.1397
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
## -0.61862995 0.09145373
## sample estimates:
## mean of x
## -0.2635881

# Two-sample t-test
p22_a <- rnorm(30, mean = 1)
p22_b <- rnorm(30, mean = 0)
t.test(p22_a, p22_b)

##
```

```
## Welch Two Sample t-test
##
## data: p22_a and p22_b
## t = 4.6558, df = 56.38, p-value = 2.005e-05
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## 0.6457107 1.6207352
## sample estimates:
## mean of x mean of y
## 0.8951521 -0.2380708
```

```
# Paired t-test
t.test(p22_a, p22_b, paired = TRUE)
```

```
##
## Paired t-test
##
## data: p22_a and p22_b
## t = 4.9689, df = 29, p-value = 2.766e-05
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
## 0.6667787 1.5996671
## sample estimates:
## mean difference
## 1.133223
```

```
# Correlation test
cor.test(p22_x, rnorm(30))
```

```
##
## Pearson's product-moment correlation
##
## data: p22_x and rnorm(30)
## t = 0.045876, df = 28, p-value = 0.9637
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## -0.3527014 0.3677899
## sample estimates:
## cor
## 0.008669408
```

```
# Chi-square test
chisq.test(matrix(c(10, 20, 30, 40), nrow = 2))
```

```
##
## Pearson's Chi-squared test with Yates' continuity correction
##
## data: matrix(c(10, 20, 30, 40), nrow = 2)
## X-squared = 0.44643, df = 1, p-value = 0.504
```

24.4 22.4 Regression

```
# --- Self-contained: regression ---
set.seed(2204)
p22_df <- tibble(x = rnorm(50), y = 2 + 3 * x + rnorm(50))
```

```

p22_fit <- lm(y ~ x, data = p22_df)
summary(p22_fit)

##
## Call:
## lm(formula = y ~ x, data = p22_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.3262 -0.6840 -0.1218  0.7488  2.0523
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.9564     0.1614   12.12 3.25e-16 ***
## x             3.0656     0.1678   18.27 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.102 on 48 degrees of freedom
## Multiple R-squared:  0.8743, Adjusted R-squared:  0.8717
## F-statistic: 333.8 on 1 and 48 DF,  p-value: < 2.2e-16
coef(p22_fit)

## (Intercept)          x
##      1.956433      3.065566
confint(p22_fit)

##              2.5 %    97.5 %
## (Intercept) 1.631879 2.280988
## x           2.728216 3.402917
predict(p22_fit, newdata = data.frame(x = 1))

##      1
## 5.022

```

24.5 22.5 Correlation

```

# --- Self-contained: correlation ---
set.seed(2205)
p22_df <- tibble(x = rnorm(50), y = rnorm(50))

cor(p22_df$x, p22_df$y) # Pearson

## [1] 0.05094286
cor(p22_df$x, p22_df$y, method = "spearman")

## [1] 0.08043217
cor(p22_df$x, p22_df$y, method = "kendall")

## [1] 0.05795918
cor(p22_df) # correlation matrix

```

```
##           x           y
## x 1.00000000 0.05094286
## y 0.05094286 1.00000000
```

24.6 22.6 Hypothesis test decision table

Question	Function
Compare one mean to a value	<code>t.test(x, mu = 0)</code>
Compare two independent means	<code>t.test(x, y)</code>
Compare paired means	<code>t.test(x, y, paired = TRUE)</code>
Compare >2 means	<code>aov(y ~ g)</code> or <code>anova(lm(y ~ g))</code>
Test normality	<code>shapiro.test(x)</code>
Test variance equality	<code>var.test(x, y)</code>
Test a proportion	<code>prop.test(x, n, p)</code>
Test association (categorical)	<code>chisq.test(table)</code>
Correlation	<code>cor.test(x, y)</code>

25 Part 23 — Random Numbers and Sampling

25.1 23.1 Setting the seed

```
# --- Self-contained: set.seed ---
set.seed(42)
rnorm(3)

## [1] 1.3709584 -0.5646982 0.3631284

set.seed(42)
rnorm(3) # same values
```

```
## [1] 1.3709584 -0.5646982 0.3631284
```

Rule: Always `set.seed()` before simulation or sampling for reproducibility.

25.2 23.2 Common generators

```
# --- Self-contained: generators ---
set.seed(2302)
rnorm(5) # N(0, 1)

## [1] -1.0347913 0.3684285 -0.9426302 -1.6902442 0.4917266

rnorm(5, mean = 10, sd = 2)

## [1] 9.277410 9.415711 10.855956 7.354555 11.898515

runif(5) # U(0, 1)

## [1] 0.74364891 0.07052087 0.32032081 0.12659006 0.54446524

runif(5, min = 0, max = 100)

## [1] 7.800757 38.153568 35.506917 8.769078 31.378870
```

```

rbinom(5, size = 10, prob = 0.3)

## [1] 5 5 5 4 5
rpois(5, lambda = 3)

## [1] 4 4 1 0 3
rexp(5, rate = 1)

## [1] 1.9382935 0.6037421 1.5950037 3.1816264 1.1153521
rt(5, df = 5)

## [1] 0.03406165 0.64742850 -1.04380143 -0.49607860 -1.50660296

```

25.3 23.3 Sampling

```

# --- Self-contained: sampling ---
set.seed(2303)
p23_v <- 1:10

sample(p23_v, size = 3)                # without replacement

## [1] 1 7 8
sample(p23_v, size = 3, replace = TRUE) # with replacement

## [1] 7 7 4
sample(p23_v)                          # permutation

## [1] 7 2 5 3 8 4 6 10 1 9
sample(c("H", "T"), size = 10, replace = TRUE, prob = c(0.5, 0.5))

## [1] "T" "H" "T" "H" "H" "T" "T" "T" "H" "T"

```

25.4 23.4 Split into train/test

```

# --- Self-contained: train/test split ---
set.seed(2304)
p23_df <- tibble(x = rnorm(100), y = rnorm(100))

p23_idx <- sample(1:nrow(p23_df), size = 0.8 * nrow(p23_df))
p23_train <- p23_df[p23_idx, ]
p23_test <- p23_df[-p23_idx, ]

nrow(p23_train); nrow(p23_test)

## [1] 80
## [1] 20

```

26 Part 24 — Vectorization vs Loops

26.1 24.1 Why vectorize

R is optimized for vector operations. Loops are slow.

```
# --- Self-contained: timing ---
set.seed(2401)
p24_v <- rnorm(1e6)

# Slow: loop
system.time({
  p24_out <- numeric(length(p24_v))
  for (i in seq_along(p24_v)) p24_out[i] <- p24_v[i]^2
})

##      user      system elapsed
##    0.072     0.004     0.076

# Fast: vectorized
system.time({
  p24_out2 <- p24_v^2
})

##      user      system elapsed
##    0.002     0.003     0.007
```

26.2 24.2 Vectorized operations

```
# --- Self-contained: vectorized operations ---
p24_a <- c(1, 2, 3)
p24_b <- c(10, 20, 30)

p24_a + p24_b          # element-wise

## [1] 11 22 33
p24_a * p24_b

## [1] 10 40 90
p24_a / p24_b

## [1] 0.1 0.1 0.1
p24_a ^ 2

## [1] 1 4 9
sqrt(p24_a)

## [1] 1.000000 1.414214 1.732051
log(p24_a)

## [1] 0.0000000 0.6931472 1.0986123
p24_a > 1.5

## [1] FALSE  TRUE  TRUE
```

26.3 24.3 Recycling

R recycles shorter vectors:

```
# --- Self-contained: recycling ---  
c(1, 2, 3) + c(10, 20) # warning: longer object length not multiple
```

```
## [1] 11 22 13
```

```
c(1, 2, 3, 4) + c(10, 20) # 11 22 13 24
```

```
## [1] 11 22 13 24
```

26.4 24.4 When to use loops

Loops are fine when: - You need to update values based on previous iterations (e.g., recursive simulation). - The computation is inherently sequential. - You use `lapply` / `purrr::map` instead.

27 Part 25 — Debugging

27.1 25.1 Errors vs warnings vs messages

- **Error:** code stops. E.g., `log("a")`.
- **Warning:** code runs but something is off. E.g., `as.numeric("abc")`.
- **Message:** informational. E.g., package loading.

```
log("a") # Error  
as.numeric("abc") # Warning: NAs introduced by coercion
```

27.2 25.2 Reading error messages

Example: Error in `x[i]` : subscript out of bounds.

- Function: `x[i]`
- Problem: `i` is larger than `length(x)` or negative invalid.

Check:

```
# --- Self-contained: check indices ---  
p25_x <- 1:5  
length(p25_x)
```

```
## [1] 5
```

```
p25_x[10] # NA
```

```
## [1] NA
```

```
# p25_x[[10]] # Error
```

27.3 25.3 Debugging tools

```
# Print statements  
print(x); cat("x =", x, "\n"); str(x); class(x); dim(x)  
  
# Traceback after error  
traceback()
```

```

# Debug a function step-by-step
debug(lm)
lm(y ~ x, data = df)
undebug(lm)

# Browser inside a function
# Add browser() where you want to pause

# try / tryCatch
try(log("a"))
result <- tryCatch(log("a"), error = function(e) NA)

```

27.4 25.4 Common checks

```

# --- Self-contained: inspect ---
p25_df <- tibble(x = c(1, 2, NA), y = c("a", "b", "c"))

str(p25_df)

## tibble [3 x 2] (S3: tbl_df/tbl/data.frame)
## $ x: num [1:3] 1 2 NA
## $ y: chr [1:3] "a" "b" "c"

class(p25_df)

## [1] "tbl_df"      "tbl"          "data.frame"

dim(p25_df)

## [1] 3 2

names(p25_df)

## [1] "x" "y"

head(p25_df)

## # A tibble: 3 x 2
##       x y
##   <dbl> <chr>
## 1     1 a
## 2     2 b
## 3    NA c

summary(p25_df)

##           x           y
##  Min.   :1.00   Length :3
## 1st Qu.:1.25   N.unique :3
##  Median:1.50   N.blank  :0
##   Mean   :1.50   Min.nchar:1
## 3rd Qu.:1.75   Max.nchar:1
##   Max.   :2.00
##   NAs    :1

sapply(p25_df, class)

```

```
##           x           y
##  "numeric" "character"
anyNA(p25_df)

## [1] TRUE
```

27.5 25.5 stopifnot

```
my_fun <- function(x) {
  stopifnot(is.numeric(x), length(x) > 0)
  mean(x)
}
my_fun(c(1, 2, 3)) # works
my_fun("a")       # error
```

28 Part 26 — Common Errors and Fixes

Error	Cause	Fix
object 'x' not found	Variable not defined or misspelled	Check spelling, define <code>x</code> first
could not find function "f"	Package not loaded or typo	<code>library(pkg)</code> or fix spelling
unexpected symbol	Syntax error (missing <code>)</code> , <code>,</code>)	Check parentheses and commas
subscript out of bounds	Index beyond vector length	Check <code>length()</code> , use <code>[</code> not <code>[[</code> for out-of-range
non-numeric argument to binary operator	Wrong type	<code>as.numeric()</code> or check <code>class()</code>
NAs introduced by coercion	Converting non-numeric string	Check the string, handle <code>NA</code>
replacement has X rows, data has Y	Length mismatch in assignment	Match lengths
object of type 'closure' is not subsettable	You called a function like a variable	Don't name variables the same as functions
cannot open file	Wrong path	Use absolute path or <code>setwd()</code>
there is no package called 'X'	Not installed	<code>install.packages("X")</code>
contrasts can be applied only to factors with 2 or more levels	Factor with 1 level	Check <code>unique(df\$var)</code> , use <code>factor()</code>
model frame and formula mismatch	Removed rows in one variable	Use <code>na.omit()</code> or <code>drop_na()</code> first
singular fit / system is computationally singular	Perfect collinearity	Remove one predictor
longer object length is not a multiple of shorter object length	Recycling	Match lengths or use <code>rep()</code>
<code>\$</code> operator is invalid for atomic vectors	Using <code>\$</code> on a vector	Use <code>[</code> or <code>[[</code>

28.1 26.1 Quick fixes

```

# --- Self-contained: fix patterns ---
# Check types
p26_x <- c(1, 2, 3)
class(p26_x)

## [1] "numeric"

# Rename variable to avoid function clash
p26_mean <- c(1, 2, 3) # good
# mean <- c(1, 2, 3)   # bad: overwrites function

# Use [[ not [ for lists when extracting elements
p26_l <- list(a = 1:3)
p26_l[["a"]] # correct: returns vector

## [1] 1 2 3

# p26_l[["b"]] # Error: subscript out of bounds

```

29 Part 27 — RStudio Shortcuts

Shortcut	Action
Ctrl+Enter	Run current line / selection
Ctrl+Shift+Enter	Run entire script
Ctrl+Alt+R	Run entire script (alt)
Ctrl+Shift+K	Knit (RMarkdown)
Ctrl+1	Focus source editor
Ctrl+2	Focus console
Alt+-	Insert <-
Ctrl+Shift+M	Insert %>%
Ctrl+Shift+A	Reformat code
Ctrl+D	Delete current line
Ctrl+Shift+D	Duplicate line
Ctrl+L	Clear console
Ctrl+Shift+C	Comment/uncomment
Ctrl+F	Find
Ctrl+Shift+F	Find in files
F1	Help for function at cursor
F2	Go to definition
Tab	Autocomplete
Ctrl+Up	Command history
Alt+Shift+K	Keyboard shortcut help

30 Part 28 — Quick Reference Table

Task	Code
Assign	x <- 5

Task	Code
Sequence	<code>1:10, seq(0, 1, by = 0.1)</code>
Repeat	<code>rep(1:3, times = 2)</code>
Vector	<code>c(1, 2, 3)</code>
List	<code>list(a = 1, b = "x")</code>
Matrix	<code>matrix(1:9, nrow = 3)</code>
Data frame	<code>data.frame(x = 1:3, y = 4:6)</code>
Tibble	<code>tibble(x = 1:3, y = 4:6)</code>
Access column	<code>df\$x</code> or <code>df[["x"]]</code>
Access element	<code>df[1, 2]</code> or <code>v[3]</code>
Filter	<code>df > filter(x > 5)</code>
Select cols	<code>df > select(a, b)</code>
Add column	<code>df > mutate(z = x + y)</code>
Sort	<code>df > arrange(x)</code>
Group + summarize	<code>df > group_by(g) > summarise(m = mean(x))</code>
Join	<code>a > inner_join(b, by = "id")</code>
Reshape long	<code>pivot_longer(cols, names_to, values_to)</code>
Reshape wide	<code>pivot_wider(names_from, values_from)</code>
Mean	<code>mean(x, na.rm = TRUE)</code>
Median	<code>median(x)</code>
SD	<code>sd(x)</code>
Quantiles	<code>quantile(x, c(.25, .75))</code>
T-test	<code>t.test(x, y)</code>
Correlation	<code>cor(x, y) / cor.test(x, y)</code>
Regression	<code>lm(y ~ x, data = df)</code>
Predict	<code>predict(fit, newdata)</code>
Set seed	<code>set.seed(42)</code>
Sample	<code>sample(x, size = n, replace = FALSE)</code>
Map	<code>map(x, f) / map_dbl(x, f)</code>
Scatter plot	<code>ggplot(df, aes(x, y)) + geom_point()</code>
Save plot	<code>ggsave("plot.png")</code>
Read CSV	<code>read_csv("file.csv")</code>
Write CSV	<code>write_csv(df, "file.csv")</code>

31 Part 29 — Theory Q&A (What to Say)

Q: What’s the difference between `<-` and `=`? A: Both assign in most contexts, but `<-` is idiomatic and preferred. Use `=` only for function arguments (`mean(x, na.rm = TRUE)`).

Q: What’s the difference between `[`, `[[`, and `$`? A: `[` returns a **subset** (vector, list, data frame). `[[` returns the **contents** of a single element (unwrapped). `$` is shorthand for `[[` on a named element.

Q: What’s the difference between a vector and a list? A: A vector holds elements of the **same type**. A list can hold elements of **different types** (including other lists).

Q: What’s the difference between `NA` and `NULL`? A: `NA` is a missing value placeholder that occupies a position. `NULL` means “no value at all” — assigning to a vector element with `NULL` removes it.

Q: What’s the difference between a factor and a character? A: Character is just text. A factor stores categorical data with predefined **levels**, which matters for regression (`lm` treats factors as dummies) and plotting.

Q: What's the difference between `&` and `&&`? A: `&` is vectorized (returns a vector). `&&` is scalar and short-circuits (returns one value, stops at first FALSE).

Q: Why use the pipe `|>`? A: It makes nested function calls readable: instead of `f(g(h(x)))`, write `x |> h() |> g() |> f()`.

Q: What is vectorization? A: Performing an operation on an entire vector at once, instead of element-by-element. It's faster and cleaner than loops in R.

Q: What is the difference between `apply`, `lapply`, `sapply`, `tapply`, `mapply`? A: `apply` works on matrix margins. `lapply` returns a list. `sapply` simplifies to a vector. `tapply` applies by group. `mapply` handles multiple inputs.

Q: How do you handle missing values? A: Detect with `is.na()`, remove with `na.omit()` or `drop_na()`, exclude in functions with `na.rm = TRUE`, or fill with `mean/median/zoo::na.locf()`.

Q: What's the difference between `data.frame` and `tibble`? A: Tibbles print nicely, don't auto-convert strings to factors, don't use row names, and allow list-columns.

Q: What's the difference between `read.csv` and `read_csv`? A: `read.csv` is base R, slower, uses `stringsAsFactors` (defaults changed in R 4.0). `read_csv` is tidyverse, faster, returns a tibble, doesn't convert strings.

32 Part 30 — The Single Most Important Idea

R is:

1. **Vectorized** — operations apply to whole vectors.
2. **Functional** — functions take inputs, return outputs.
3. **Pipe-friendly** — chains read top-to-bottom.
4. **Type-aware** — every object has a class (`numeric`, `character`, `factor`, `tbl`, ...).
5. **Subsettable** — `[], [[, $` extract pieces.

If you know these five things and can look up functions, you can do anything in R.

The pipeline for any R task:

1. **Load** the data (`read_csv`, `readRDS`).
2. **Inspect** (`str`, `glimpse`, `summary`, `head`).
3. **Clean** (`filter`, `mutate`, `drop_na`).
4. **Transform** (`group_by`, `summarise`, `pivot_*`).
5. **Analyze** (`lm`, `t.test`, `glm`).
6. **Visualize** (`ggplot` or base `plot`).
7. **Report** (`kable`, `glue`, `sprintf`).

If you can do these seven steps, you can complete any R task.

End of cheat sheet. Good luck in the competition. ““

32.1 What's inside and why

Part	Covers
0	Five survival rules
1–2	Packages, help

Part	Covers
3–4	Variables, assignment, data types (numeric, integer, character, logical, complex)
5	Vectors, matrices, arrays, lists, data frames, tibbles
6	Indexing with <code>[]</code> , <code>[[</code> , <code>\$</code> , logical/negative/name indexing
7	Arithmetic, comparison, logical, membership, pipes, <code>Inf/NaN/NA/NULL</code>
8	Pipes in depth + placeholder + anonymous functions
9	Functions: <code>define</code> , default args, <code>...</code> , anonymous, scope
10	<code>if/else</code> , <code>for</code> , <code>while</code> , <code>break/next</code> , apply family
11	Strings: base, <code>grep/gsub</code> , <code>stringr</code> , <code>sprintf</code> , <code>glue</code>
12	Factors: <code>levels</code> , <code>relevel</code> , tables, <code>forcats</code>
13	Dates: <code>as.Date</code> , components, arithmetic, <code>lubridate</code>
14	Missing values: <code>detect</code> , <code>remove</code> , <code>fill</code> , <code>NA</code> vs <code>NULL</code> vs <code>NaN</code>
15	Read/write CSV, Excel, RDS, RData
16	Full dplyr toolkit: <code>filter</code> , <code>select</code> , <code>mutate</code> , <code>arrange</code> , <code>group_by</code> , <code>summarise</code> , <code>across</code> , <code>slice</code> , <code>pull</code> , <code>relocate</code>
17	tidyr: <code>pivot_longer</code> , <code>pivot_wider</code> , <code>separate</code> , <code>unite</code> , <code>complete</code> , <code>fill</code>
18	Joins: <code>inner</code> , <code>left</code> , <code>right</code> , <code>full</code> , <code>anti</code> , <code>semi</code>
19	purrr: <code>map</code> , <code>map_dbl</code> , <code>map2</code> , <code>pmap</code> , <code>walk</code>
20	Base plots: <code>plot</code> , <code>hist</code> , <code>boxplot</code> , <code>barplot</code> , <code>layout</code> , <code>save</code>
21	ggplot2: <code>scatter</code> , <code>line</code> , <code>hist</code> , <code>boxplot</code> , <code>bar</code> , <code>facets</code> , <code>saving</code>
22	Statistical functions: descriptive, distributions, tests, regression, correlation
23	Random numbers, sampling, <code>seed</code> , train/test split
24	Vectorization vs loops + recycling
25	Debugging tools: <code>traceback</code> , <code>debug</code> , <code>tryCatch</code> , <code>stopifnot</code>
26	15 common errors with fixes
27	RStudio shortcuts
28	Quick reference table
29	Theory Q&A
30	Single most important idea